HRMS — Technical Blueprint

Production-ready plan for a multi-tenant HR Management System
Employees · Attendance · Leave · Payroll · Documents · Recruitment · Performance · Training · Expenses · Reports

#	Document	File
—	Index & Quick Reference	README.md
00	Product Overview, Requirements & Role Model	00-overview-and-roles.md
01	Core Modules (1–8)	01-modules-core.md
02	Talent & Money Modules (9–16)	02-modules-talent.md
03	Platform Modules (17–23) & Reports Catalog	03-modules-platform-and-reports.md
04	Database Schema & Multi-Tenant Model	04-database.md
05	Application Architecture	05-architecture.md
06	UI/UX Screen Specifications	06-ui-ux.md
07	Workflows, Notifications & Automation	07-workflows-and-automation.md
08	API Specification	08-api.md
09	Security, Privacy & Data Protection	09-security.md
10	Roadmap, Testing, Deployment & Build Order	10-roadmap-testing-deployment.md

Generated 22 August 2026 · 12 documents · Single source of truth for implementation

README.MD

HRMS Blueprint — Index

Single source of truth for building the HRMS. Read order = numeric order. Where docs disagree: `04-database.md` wins on data shapes; `00-overview-and-roles.md` wins on scope/roles; `08-api.md` wins on endpoint naming.

Doc	Contents
00-overview-and-roles.md	PRD, personas, problems, goals, architecture + stack decision, feature hierarchy, role & permission model, user-role matrix
01-modules-core.md	Modules 1–8: Auth, Employees, Company/Org, Departments/Designations, Attendance, Shifts, Leave, Holidays
02-modules-talent.md	Modules 9–16: Payroll, Documents, Recruitment, Onboarding, Performance, Training/LMS, Expenses, Self-Service
03-modules-platform-and-reports.md	Modules 17–23: dashboards, notifications, reports catalog (R1–R16), offboarding, audit logs, settings catalog
04-database.md	**Canonical schema** (~55 tables), relationships, multi-tenant model (company_id + Prisma extension + RLS), seed data, migration policy
05-architecture.md	System diagram, frontend/backend structure, auth flow, storage, notification service, jobs/cron table, integration seams, logging, full folder tree
06-ui-ux.md	Design system, global UI conventions, every screen spec (public/employee/manager/HR/admin), composite screens, notification UX
07-workflows-and-automation.md	State machines for onboarding/attendance/leave/payroll/offboarding/recruitment/training + notification & automation catalog
08-api.md	Full REST endpoint catalog per module with permissions, requests, responses, error rules
09-security.md	Threat model, authn/authz, RLS, file security, input handling, CSRF, rate limits, audit, privacy, backups, retention
10-roadmap-testing-deployment.md	MVP rationale, milestone roadmap, testing strategy + HR test cases, deployment/ops, checklists, **Build Order**

Final-deliverables map

1. Product requirements document → 00 §1–3
2. Feature hierarchy → 00 §5
3. User-role matrix → 00 §6.4
4. Complete database schema → 04 §2
5. Entity relationship explanation → 04 §3
6. Application architecture → 05 §1–9
7. Folder structure → 05 §10
8. Page/screen map → 06 §3–7
9. API map → 08
10. Workflow diagrams (text) → 07 §1
11. Security architecture → 09 (+ 04 §4)
12. MVP roadmap → 10 §1–2 (Phase 1)
13. Development roadmap → 10 §2
14. Testing checklist → 10 §3
15. Deployment checklist → 10 §4–5
16. Future enhancement roadmap → 10 §2 (Phase 2/3) + 00 §3

Canonical quick reference

- Stack: Next.js 15 (App Router, TS strict) · Tailwind + shadcn/ui · TanStack Query/Table · react-hook-form + zod · REST `/api/v1` · PostgreSQL 16 + Prisma · Auth.js v5 (argon2id, JWT cookie) · BullMQ + Redis worker · S3-compatible storage · Resend/Mailpit ·

@react-pdf/renderer · Docker Compose · GitHub Actions.

- Roles: `super_admin`, `hr_admin`, `manager`, `employee`. Permissions: `module.action` with actions `view_own|view_team|view_all|create|edit|delete|approve|export|manage`.
- Tenancy: shared schema, `company_id` everywhere, tenant-scoped Prisma extension + Postgres RLS (`app.company_id` via SET LOCAL).
- Phases: P1 core HR ops (no payroll) · P2 payroll/documents/expenses/recruitment/boarding/performance/reports · P3 LMS/analytics/integrations/PWA/multi-tenant GA.

HRMS Blueprint — Product Overview, Requirements & Role Model

> Document 00 of 11. This doc set is the single source of truth for building the HRMS. Canonical data shapes live in `04-database.md`. Canonical decisions (stack, tables, enums, routes, phases) are repeated consistently across all docs; where docs disagree, `04-database.md` wins for data and this doc wins for scope/roles.

---

1. Product Requirements Document (PRD)

1.1 Purpose

A single web platform where a company manages the entire employee lifecycle: hiring, onboarding, daily attendance, shifts, leave, payroll, documents, performance, training, expenses, internal communication, offboarding, and reporting. It replaces spreadsheets, email threads, and disconnected point tools with one permission-controlled, auditable system.

The system ships as a **single-company application** but is **multi-tenant-ready from day 1**: every tenant-scoped table carries `company_id`, and authorization is built on tenant isolation + permissions, so supporting many companies later requires onboarding flows — not a rewrite.

1.2 Target users / personas

Persona	Role in system	What they need
Founder / IT owner	`super_admin`	Platform configuration, company setup, roles/permissions, audit visibility, data safety
HR generalist / HR head	`hr_admin`	Employee master data, attendance/leave administration, payroll runs, recruitment, documents, reports
Team lead / department head	`manager`	Team attendance, leave approvals, corrections approvals, team performance, team reports
Every staff member	`employee`	Self-service: check-in/out, apply leave, view payslips/documents, submit expenses, training, own profile

One person can hold multiple roles (e.g., an HR head is also a manager of the HR team) via `user_roles`.

1.3 Business problems solved

1. **Scattered records** — employee data in spreadsheets, contracts in email, no single profile. → One employee master with documents attached.
2. **Untrusted attendance** — manual registers, no late/overtime rules. → Timestamped check-in/out with shift rules, corrections with approval trails.
3. **Leave chaos** — balances tracked by hand, disputes about carry-forward. → Policy-driven accruals, live balances, approval workflow, full history.
4. **Slow, error-prone payroll** — manual LOP counting, no payslip archive. → Payroll runs computed from attendance/leave snapshots, approved, published as PDFs.
5. **No manager visibility** — managers can't see team status without asking HR. → Team dashboards scoped to direct reports.
6. **Zero auditability** — no record of who changed a salary or approved a leave. → Append-only audit logs on all sensitive actions.
7. **Compliance risk** — expiring contracts/documents unnoticed. → Expiry tracking with automated reminders.

1.4 Goals

- Every HR workflow (listed in §2) executable end-to-end inside the system with correct permissions.
- An employee's day-1 needs (check-in, leave, profile) usable on a phone browser.
- HR can answer "who was absent last month?", "what did we pay in March?", "whose contract expires in 30 days?" in under a minute.
- All sensitive reads/writes are permission-checked server-side and audit-logged.

- A second company can be onboarded with configuration only (no schema or code changes).

1.5 Non-goals

- Not a public job board, benefits marketplace, or background-check provider.
- No country-specific statutory tax engine hard-coded in core (statutory items are configurable salary components; a country pack can be layered later).
- No native mobile apps (responsive web first; PWA in Phase 3).
- No custom workflow builder / BPM engine — approval flows are fixed single-level (manager → HR override) until proven insufficient.
- No AI features in scope for v1–v3.

1.6 Success metrics

- 100% of active employees checked in via the system within 2 weeks of pilot.
- Leave requests resolved (approved/rejected) in < 48h median.
- Payroll run for the pilot company completed in < 1 day of effort (Phase 2).
- Zero cross-tenant data reads in isolation tests (continuous, automated).
- < 1% attendance records requiring manual correction after month 2.

---

2. Core workflows (summary)

Full state machines with side effects live in `07-workflows-and-automation.md`.

- **Employee onboarding** — A hired candidate's accepted offer converts into an `employees` row (status `onboarding`), HR collects documents, an onboarding checklist (from a template) is executed by HR/IT/manager/new hire, and on completion the employee becomes `active` with a user account invited.
- **Attendance** — Employee checks in/out (punches accumulate); a nightly job computes the daily `attendance_records` row (status, worked minutes, late minutes, overtime) against the assigned shift; discrepancies are fixed via `attendance_corrections` with manager approval; the month is locked before payroll.
- **Leave** — Employee submits a `leave_requests` row against a live balance; manager approves/rejects (HR can override); approval deducts `leave_balances`, cancellation restores it; every transition notifies the parties.
- **Payroll** — HR opens a `payroll_runs` period; the system snapshots attendance/leave, computes LOP, prorates salary components, produces payslips; HR reviews, `hr_admin` approves; payslip PDFs are generated and published to employees; run is marked paid.
- **Offboarding** — Resignation is submitted and approved; notice period and last working day are computed; exit checklist and asset clearance run; final settlement inputs hand off to payroll; on completion the user account is disabled and the employee becomes `exited`.
- **Recruitment** — HR publishes a `job_postings` row; candidates are added; each `applications` row moves applied → screening → interview → offer → hired/rejected; interviews collect structured feedback; an accepted offer triggers conversion to employee (onboarding workflow).
- **Training** — HR publishes a course with lessons; employees are enrolled (self or assigned); progress is tracked per lesson; completion issues a certificate.

---

3. MVP scope vs future scope

Detailed milestone plan: `10-roadmap-testing-deployment.md`.

Phase 1 — MVP (launchable HRMS)

Auth + invites, permission-based RBAC, company/org setup, locations, departments, designations, employee master + profiles, shifts (definitions + assignment), attendance (web check-in/out, punches, nightly calculation, corrections), leave (types, policies, balances, requests, approvals), holidays, employee self-service, manager team view + approvals, basic HR dashboard, in-app + email notifications, audit logs, basic system settings.

Deliberately NOT in MVP: payroll, recruitment, onboarding/offboarding modules, performance, LMS, expenses, document-expiry automation, advanced analytics/report builder, PWA/push, custom roles, SSO, biometric integration. (Rationale per item in `10-roadmap-testing-deployment.md` §1.)

Phase 2

Payroll (components, structures, runs, payslips), employee documents, expenses & reimbursements, recruitment, onboarding, offboarding, performance management, advanced shift rostering, reports + exports.

Phase 3

Training/LMS, advanced analytics, automation rules, integrations (biometric devices, accounting, calendar, Slack), PWA + web push, multi-tenant self-serve signup, custom roles, SSO.

---

4. Recommended architecture

4.1 Options compared

Option	Strengths	Weaknesses
(a) Modular monolith — Next.js full-stack	One language/repo/deploy; fastest iteration; server components + REST API in one app; module boundaries enforced by convention allow later extraction	Requires discipline to keep module boundaries; worker runs as a second process
(b) NestJS API + React SPA	Strong DI/module system; clean API separation	Two apps to build/deploy/version; duplicated types unless extra tooling; slower for a small team
(c) BaaS (Supabase)	Auth/RLS/storage out of the box; very fast start	Payroll math, workflow state machines, PDF generation, and scheduled jobs need a real server anyway; vendor lock-in on auth; harder local testing

Recommendation: (a) modular monolith on Next.js. A small team ships one TypeScript codebase; domain logic lives in `src/modules/*` behind service functions, so Payroll/Recruitment/LMS bolt on without touching core; if scale ever demands it, a module extracts into its own service because nothing else touches its tables directly.

4.2 Why this scales to multi-tenant without rewrite

- `company_id` is on every tenant table from the first migration; all queries pass through a tenant-scoped Prisma client extension.
- PostgreSQL RLS policies (session variable `app.company_id`) provide defense-in-depth beneath the app layer.
- Authorization is permissions + data-scope tiers (own/team/company/platform) — nothing assumes "there is only one company."
- Becoming true SaaS later = add company signup/provisioning + billing, not schema surgery.

4.3 Technology stack (canonical)

Layer	Choice	Why	Alternative considered
Framework	Next.js 15 (App Router, TypeScript strict)	One deployable for UI + API; RSC for fast reads; huge ecosystem	NestJS + React SPA
UI	Tailwind CSS + shadcn/ui, lucide-react, Recharts	Professional look fast; accessible primitives; no design-system tax	MUI, Ant Design
Data fetching / tables / forms	TanStack Query, TanStack Table, react-hook-form + zod	Server-state caching; headless tables; shared client/server validation	SWR, Formik
API	REST route handlers `/api/v1/*`	Simple, cacheable, testable; no GraphQL complexity needed	GraphQL, tRPC
Database	PostgreSQL 16 + Prisma ORM (UUID PKs, prisma migrate)	Relational integrity for HR data; RLS support; typed queries	MySQL, Drizzle

Layer	Choice	Why	Alternative considered
Auth	Auth.js v5 credentials (argon2id), JWT session httpOnly cookie	Invite-only workforce auth; stateless sessions; SSO pluggable later	Clerk, Supabase Auth
AuthZ	Permission RBAC (`requirePermission()`) + tenant-scoped Prisma extension + Postgres RLS	Permission-based, tenant-safe, defense-in-depth	CASL, hard-coded roles
Files	S3-compatible storage (S3/R2 prod, MinIO dev), presigned URLs	Private-by-default documents; no server file handling	Local disk, DB blobs
Jobs	BullMQ + Redis, separate worker process	Reliable retries; cron repeatables for accruals/reminders	pg-boss, in-process cron
Email	Provider-agnostic mailer (Resend prod, Mailpit dev)	Swappable; local testing	SMTP direct
PDF	@react-pdf/renderer in worker	Payslips/certificates server-side, off request path	Puppeteer
Deploy	Docker Compose (web, worker, postgres, redis, minio); GitHub Actions CI/CD	Reproducible; runs on any VPS/Render/Fly	Vercel + managed addons

5. Feature hierarchy (all 23 modules)

Legend: **P1** = MVP, **P2**, **P3**.

1. **Authentication & Security** — invite-only accounts P1; login/logout P1; forgot/reset password P1; session management P1; account lockout P1; rate limiting P1; SSO (Google) P3; MFA P3
2. **Employee Management** — employee CRUD + profile P1; manager assignment P1; bank accounts P2 (payroll dependency); emergency contacts P1; employment types/status lifecycle P1; bulk import P2
3. **Company & Organization** — company profile & settings P1; org defaults (timezone, currency, week-offs) P1; multi-company provisioning P3
4. **Departments & Designations** — department CRUD + head P1; designation CRUD + level P1; department tree (nested) P3
5. **Attendance** — web check-in/out + punches P1; nightly calculation P1; corrections + approval P1; month lock P1; overtime capture P1 (approval P1, payout P2); biometric ingestion P3
6. **Shift Management** — shift definitions (times, grace, thresholds, week-offs) P1; employee assignment with effective dates P1; rotation/rostering P2
7. **Leave Management** — leave types P1; policies (accrual, carry-forward, proration, sandwich rule) P1; balances P1; requests + approvals P1; half-day leave P1; comp-off P2
8. **Holiday Management** — holiday calendar CRUD P1; location-specific holidays P1; optional/floating holidays P2
9. **Payroll** — salary components P2; structures P2; employee salary assignment P2; payroll runs P2; payslips + PDF publish P2; final settlement P2; statutory country pack P3
10. **Employee Documents** — categories P2; upload/download (presigned) P2; expiry tracking P2; expiry automation P2; e-sign P3
11. **Recruitment** — job postings P2; candidates P2; application pipeline P2; interviews + feedback P2; offers P2; convert-to-employee P2; careers page P3
12. **Onboarding** — task templates P2; per-hire checklists P2; multi-assignee tasks P2
13. **Performance Management** — cycles P2; goals P2; self + manager reviews P2; ratings P2; 360° feedback P3
14. **Training/LMS** — courses + lessons P3; enrollments + progress P3; certificates P3; assessments/quizzes P3
15. **Expenses & Reimbursements** — categories P2; claims with receipts P2; approval P2; reimbursement payout (payroll or direct) P2
16. **Employee Self-Service** — dashboard P1; own attendance/leave/profile P1; payslips P2; documents P2; expenses P2; training P3
17. **Manager Dashboard** — team overview P1; pending approvals P1; team attendance P1; team performance P2; team reports P2
18. **HR Dashboard** — headcount & presence KPIs P1; pending actions P1; charts P1 (basic) / P2 (extended)
19. **Notifications** — in-app P1; email P1; digests P2; web push P3; preferences P3
20. **Reports & Analytics** — attendance/leave reports P1 (basic lists); full catalog + exports P2; analytics dashboards P3
21. **Offboarding** — resignation + approval P2; exit checklist P2; asset clearance P2; settlement handoff P2; account deactivation P2 (deactivation itself exists P1 via employee status)

22. **Audit Logs** — append-only log P1; viewer + filters P1; retention policy P2
23. **System Settings** — company settings P1; attendance/leave rules P1; password policy P1; notification toggles P2

---

6. Role & permission system

6.1 System roles

Role	Scope	Summary
`super_admin`	platform	Everything, across companies (when multi-tenant). Company provisioning, global settings, all audit logs. Not part of approval chains.
`hr_admin`	company	Full HR administration inside own company: employees, attendance, leave, payroll, recruitment, documents, reports, settings, roles assignment. Can override any approval.
`manager`	team	Direct reports only: view team attendance/leave/profiles (work data, not salary), approve leave/corrections/expenses/overtime, team reports.
`employee`	own	Self-service only: own profile, attendance, leave, payslips, documents, expenses, training, notifications.

Roles are **seeded rows**, not code constants. A user gets roles via `user_roles`; a role gets capabilities via `role_permissions`. Custom roles (Phase 3) are just new rows — zero code change, which is the point of permission-based design.

6.2 Permission architecture

- Permission = string `module.action`, e.g. `leave.approve`, `employee.view_all`, `payroll.manage`.
- Action vocabulary (closed set): `view_own`, `view_team`, `view_all`, `create`, `edit`, `delete`, `approve`, `export`, `manage`.
- `manage` = module administration (configure types/policies/templates/settings for that module); it does not imply the other actions — grants are explicit.
- Enforcement is server-side only: every API route declares its required permission; UI uses the same permission set to show/hide navigation and buttons (cosmetic, never trusted).
- **Never** `if (role === 'hr_admin')` in feature code. Always `can(user, 'module.action')`.

6.3 Data-scope tiers (combine with permissions)

Tier	Meaning	Enforced by
own	rows where `employee_id` = caller's employee id (or `user_id` = caller)	service-layer filter
team	rows of employees whose `manager_id` = caller's employee id	service-layer filter
company	all rows in caller's `company_id`	tenant-scoped Prisma client + RLS
platform	across companies	`super_admin` only, explicit bypass client

`view_own` / `view_team` / `view_all` select the tier; tenant isolation always applies underneath (a manager with `view_team` still can't cross companies). Object-level rules add a third layer: e.g., approver ≠ requester, manager approves only direct reports.

6.4 Default role → permission matrix (all 23 modules)

Legend: **O**=view_own, **T**=view_team, **A**=view_all, **C**=create, **E**=edit, **Ap**=approve, **D**=delete, **X**=export, **M**=manage. `super_admin` holds every permission platform-wide and is omitted from cells (always: all).

People & org

Module	employee	manager	hr_admin
2 Employee Management	O, E(own limited fields)	T	A, C, E, D, X
3 Company & Organization	O(view basics)	O	A, E, M
4 Departments & Designations	O(view)	O(view)	A, C, E, D, M

Time

Module	employee	manager	hr_admin
5 Attendance	O, C(check-in/out, correction request)	T, Ap(corrections)	A, C(manual entry), E, Ap, X, M(lock)
6 Shift Management	O(view own shift)	T(view)	A, C, E, D, M
7 Leave Management	O, C(request), E(cancel own)	T, Ap	A, C(on behalf), E, Ap, X, M(types/policies)
8 Holiday Management	O(view)	O(view)	A, C, E, D, M

Money & talent

Module	employee	manager	hr_admin
9 Payroll	O(own payslips)	—	A, C, E, Ap, X, M
10 Employee Documents	O, C(upload own where allowed)	T(view work docs)	A, C, E, D, X, M(categories)
11 Recruitment	— (referrals P3)	T(interviewer: view assigned, C feedback)	A, C, E, D, X, M
12 Onboarding	O(own tasks, complete)	T(assigned tasks)	A, C, E, M(templates)
13 Performance Management	O, C(self review, goals)	T, C(manager review), Ap(goals)	A, C, E, X, M(cycles)
14 Training/LMS	O, C(self-enroll)	T(view progress)	A, C, E, D, X, M(courses)
15 Expenses & Reimbursements	O, C, E(draft)	T, Ap	A, Ap(override), E, X, M(categories, mark reimbursed)

Platform

Module	employee	manager	hr_admin
16 Employee Self-Service	O (composite of own-tier grants)	O	O
17 Manager Dashboard	—	T	A(view as any manager)
18 HR Dashboard	—	—	A
19 Notifications	O, E(mark read)	O	O + C(announcements), M
20 Reports & Analytics	—	T(team reports), X(team)	A, X, M
21 Offboarding	O, C(resignation)	T, Ap(resignation of reports)	A, C, E, Ap, M
22 Audit Logs	—	—	A(company), X
23 System Settings	—	—	E, M (company scope; global scope = super_admin)

Seed data must materialize this matrix into `permissions` + `role_permissions` rows (see `04-database.md` §6).

Decisions made in this document

- Product name placeholder is "HRMS"; branding is a launch-time concern, not a build blocker.
- `manage` action defined as module configuration and explicitly non-hierarchical (no implied grants).
- Managers see team **work** data only — never salary/payslips/bank details (field-level rule, restated in `09-security.md`).
- Recruitment interviewer capability is modeled as manager-tier permissions on assigned interviews, not a fifth role.
- Success metrics chosen for a pilot company of roughly 20–200 employees; thresholds are configuration, not code.

HRMS Blueprint — Core Modules (1–8)

> Document 01 of 11. Covers modules: 1 Authentication & Security, 2 Employee Management, 3 Company & Organization, 4 Departments & Designations, 5 Attendance, 6 Shift Management, 7 Leave Management, 8 Holiday Management. Data shapes: `04-database.md` . Endpoints: `08-api.md` . Screens: `06-ui-ux.md` . Workflows: `07-workflows-and-automation.md` .

Module 1 — Authentication & Security (Phase 1)

Purpose

Control who can enter the system and prove who they are. Accounts are **invite-only** — there is no public signup; HR creates an employee, which triggers a user invite.

Features

- Invite flow: HR creates employee → optional user account → invite email with single-use token (P1)
- Accept invite: set password, activate account (`users.status` invited → active) (P1)
- Login / logout with email + password (argon2id) (P1)
- Forgot / reset password (single-use, TTL-bound, hashed-at-rest tokens) (P1)
- JWT session in httpOnly Secure SameSite=Lax cookie; sliding expiry (P1)
- Account lockout after repeated failures + progressive delay (P1)
- Rate limiting on auth endpoints (P1)
- "Logout everywhere" via session-version claim bump (P1)
- Google SSO, MFA (P3)

User roles involved

- All roles: login, logout, reset own password.
- `hr_admin` : trigger/resend/revoke invites (via Employee Management).
- `super_admin` : disable any user, global password policy.

Main screens

`/login` , `/forgot-password` , `/reset-password` , accept-invite screen (reuses reset-password UI with invite token).

Main actions

login, logout, request-reset, reset, accept-invite, resend-invite (HR), disable-user (HR/admin).

Approval workflows

None. Security-relevant events (login failure, lockout, password change, invite issued) are audit-logged.

Required database entities

`users` , `password_reset_tokens` , `roles` , `permissions` , `role_permissions` , `user_roles` , `audit_logs` .

Dependencies

None (foundation). Every other module depends on it.

Module 2 — Employee Management (Phase 1)

Purpose

The employee master: one profile per person holding identity, job, org placement, and lifecycle status. Everything else (attendance, leave, payroll, documents) hangs off `employees.id`.

Features

- Employee CRUD with soft delete (P1)
- Profile sections: personal, contact, job (department/designation/location/manager/shift), employment type, dates (join, probation end, confirmation), status lifecycle (P1)
- Manager assignment (self-referential `manager_id`) (P1)
- Emergency contacts (P1)
- Bank accounts (P2 — payroll dependency; masked display, restricted access)
- User-account linking + invite from employee record (P1)
- Employee code auto-generation, unique per company (P1)
- Status lifecycle: `onboarding → active → on_notice → exited` (P1; `on_notice` / `exited` transitions driven by Offboarding in P2, manual in P1)
- Bulk CSV import (P2)

User roles involved

- `employee`: view own full profile; edit a limited self-service field set (phone, address, emergency contacts, photo).
- `manager`: view direct reports' work profiles (no salary/bank/ID numbers).
- `hr_admin`: full CRUD, status transitions, exports.

Main screens

`/hr/employees` (list), `/hr/employees/[id]` (detail with tabs: Overview, Job, Documents P2, Attendance, Leave, Salary P2, History), `/me/profile`.

Main actions

create, edit, change-status, assign-manager, assign-shift, invite-user, deactivate (disables `users` row too), export CSV.

Approval workflows

None in-module. Self-service edits to non-sensitive fields apply immediately; sensitive identity changes (name, DOB, IDs) are HR-only. (A field-change-approval flow is deliberately out of scope — HR performs sensitive edits.)

Required database entities

`employees`, `emergency_contacts`, `employee_bank_accounts`, `users`, `departments`, `designations`, `locations`, `audit_logs`.

Dependencies

Company & Organization (company, locations), Departments & Designations, Authentication (user linking).

Module 3 — Company & Organization (Phase 1)

Purpose

The tenant root. Holds company identity and org-wide defaults every module reads: timezone, currency, working week, leave-year start.

Features

- Company profile: name, legal name, logo, address, contact (P1)
- Org defaults: timezone, currency (char(3)), date format, default week-off pattern, leave year start month (P1)
- Locations CRUD (offices/branches; each employee belongs to one) (P1)
- Company provisioning for new tenants (P3 — until then a single seeded company)

User roles involved

- `hr_admin` : edit company profile and defaults (company scope).
- `super_admin` : create companies, global settings.
- All: read basics (name/logo in shell).

Main screens

`/admin/company` , `/admin/locations` .

Main actions

edit-company, upload-logo, create/edit/delete location.

Approval workflows

None. All changes audit-logged.

Required database entities

`companies` , `locations` , `system_settings` .

Dependencies

None (root). Everything depends on it via `company_id` .

Module 4 — Departments & Designations (Phase 1)

Purpose

Org structure (where you sit) and job title/level (what you are). Both are reporting dimensions for every report and dashboard.

Features

- Department CRUD: name, code (unique per company), optional department head (employee) (P1)
- Designation CRUD: title, code, level (int, for ordering/grading) (P1)
- Soft delete with guard: cannot delete while active employees are assigned (P1)
- Nested department tree (P3)

User roles involved

- `hr_admin` : full CRUD.
- `manager` / `employee` : read (their own placement, org directory).

Main screens

`/hr/departments` (departments + designations tabs; also linked from `/admin`).

Main actions

create, edit, soft-delete, assign-head.

Approval workflows

None.

Required database entities

`departments` , `designations` , `employees` (head reference, assignment).

Dependencies

Company & Organization. Employee Management consumes both.

Module 5 — Attendance (Phase 1)

Purpose

Trustworthy daily presence records: raw punches, computed daily status, correction flow, and a month lock that makes data payroll-safe.

Features

- Web check-in / check-out (multiple punch pairs per day allowed → breaks) (P1)
- `attendance_punches` : immutable raw events (`in` / `out` , timestamp, source `web|mobile|manual|biometric`) (P1; biometric source arrives P3)
- `attendance_records` : one row per employee per date — computed status `present|absent|half_day|on_leave|holiday|week_off` , worked minutes, late minutes, overtime minutes (P1)
- Nightly calculation job: builds/updates records for the previous day from punches + shift + approved leave + holidays + week-offs (P1)
- Rules (canonical): late if first check-in > shift.start_time + grace_minutes; half_day if worked < half_day_threshold_minutes; overtime = worked minutes beyond shift duration, payable only when approved (P1 capture/approve; P2 payout via payroll)
- Corrections (regularization): employee raises `attendance_corrections` (missed punch, wrong status) → manager approves → record recomputed (P1)
- Manual entry by HR (source `manual` , audit-logged) (P1)
- Month lock: HR locks a month per company; locked days reject punches edits and corrections (P1)
- Missing check-out detection + notification (P1)

User roles involved

- `employee` : check-in/out, view own calendar/summary, raise corrections.
- `manager` : team day/month views, approve/reject corrections of direct reports.
- `hr_admin` : all views, manual entries, force-approve, lock month, export.

Main screens

`/me/attendance` (today card + month calendar), `/team/attendance` (team day grid + pending corrections), `/hr/attendance` (company view, filters, lock control).

Main actions

check-in, check-out, request-correction, approve/reject-correction, manual-entry, lock-month, export.

Approval workflows

Correction: employee submits (`pending`) → manager of the employee approves/rejects (`approved|rejected`); employee may `cancel` while pending; `hr_admin` may approve/override any. Approval triggers recomputation of that day's record. Locked month blocks the whole flow (422 BUSINESS_RULE).

Required database entities

`attendance_punches` , `attendance_records` , `attendance_corrections` , `shifts` , `employee_shifts` , `holidays` , `leave_requests` (read), `employees` .

Dependencies

Shift Management (rules), Leave Management (on_leave status), Holiday Management, Employee Management. Payroll (P2) consumes locked months.

Module 6 — Shift Management (Phase 1 basic, Phase 2 rostering)

Purpose

Define working-time rules that attendance calculation evaluates against, and assign them to employees over time.

Features

- Shift definitions: name, code, start_time, end_time (overnight-capable), grace_minutes, half_day_threshold_minutes, break_minutes (unpaid, deducted from worked), week-off pattern (array of weekdays) (P1)
- Default company shift (seeded; new employees auto-assigned) (P1)
- Assignment with effective dates: `employee_shifts` (employee, shift, effective_from, effective_to nullable) — history preserved, one active assignment per employee per date (P1)
- Rotating rosters / bulk assignment calendar (P2)

User roles involved

- `hr_admin` : CRUD shifts, assign employees.
- `manager` : view team shifts.
- `employee` : view own shift.

Main screens

Shift list + form inside `/hr/attendance` settings area; assignment from employee detail Job tab.

Main actions

create/edit/delete shift, assign-shift (with effective_from), end-assignment.

Approval workflows

None. Assignment changes audit-logged (they change pay-relevant attendance math).

Required database entities

`shifts` , `employee_shifts` , `employees` .

Dependencies

Company & Organization (defaults). Attendance depends on this module.

Module 7 — Leave Management (Phase 1)

Purpose

Policy-driven leave: types, accrual rules, live balances, and an approval workflow employees trust.

Features

- Leave types: name, code, paid/unpaid flag, color (P1). Seeded defaults: Casual, Sick, Earned/Privilege, Unpaid (LWP); Comp-off (P2).
- Leave policies per type: accrual frequency (`monthly|yearly|none`), accrual amount, max carry-forward, max negative balance (default 0), min notice days, max consecutive days, applicable after probation flag, sandwich rule flag (P1)
- Sandwich rule (configurable, default off): holidays/week-offs falling **inside** a leave span count as leave days; adjacent ones never count (P1)
- Balances: `leave_balances` per (employee, leave_type, year) — opening, accrued, used, carried_forward, current (P1)
- Monthly accrual cron per policy; proration for mid-year joiners (join month counts if joined on/before the 15th — company setting) (P1)
- Year-end carry-forward job (caps applied per policy) (P1)
- Requests: date range, half-day flag (first/second half) for single-day requests, reason, attachment (sick note) optional (P1)
- Working-days computation for a request: excludes holidays/week-offs unless sandwich rule applies (P1)

- Overlap prevention against existing pending/approved requests (P1)
- Cancellation: pending → by employee; approved → allowed before start date (restores balance), else HR only (P1)
- HR adjustment (manual credit/debit with reason, audit-logged) (P1)

User roles involved

- `employee` : view balances/history, submit, cancel.
- `manager` : approve/reject direct reports' requests, team leave calendar.
- `hr_admin` : everything, on-behalf submission, adjustments, policy management.

Main screens

`/me/leave` (balances cards + request list + apply dialog), `/team/leave-approvals` , `/hr/leave` (all requests, balances, types/policies settings).

Main actions

apply, approve, reject, cancel, adjust-balance, manage-types, manage-policies.

Approval workflows

Request: `pending` → manager approve (`approved` , balance deducted) | reject (`rejected`) | employee cancel (`cancelled`). Approved → `cancelled` (before start, or HR override) restores balance. Approver must not be the requester; `leave.approve` + team scope required (HR bypasses team scope). Balance sufficiency checked at submit **and** re-checked at approval.

Required database entities

`leave_types` , `leave_policies` , `leave_balances` , `leave_requests` , `holidays` , `employees` , `notifications` , `audit_logs` .

Dependencies

Holiday Management (day counting), Employee Management (probation, manager), Attendance (marks `on_leave`), Notifications.

Module 8 — Holiday Management (Phase 1)

Purpose

Company holiday calendar consumed by attendance calculation, leave day-counting, and dashboards.

Features

- Holiday CRUD: name, date, optional location_id (null = company-wide) (P1)
- Year view + upcoming-holidays widget (P1)
- Optional/floating holidays (employee picks N from a pool) (P2)
- Bulk import for a year (P2)

User roles involved

- `hr_admin` : CRUD.
- All: view calendar.

Main screens

Holidays tab under `/hr/leave` (admin) ; upcoming-holidays widget on `/dashboard` .

Main actions

create, edit, delete, filter-by-location/year.

Approval workflows

None.

Required database entities

`holidays`, `locations`.

Dependencies

Company & Organization (locations). Attendance and Leave consume it.

Decisions made in this document

- Multiple punch pairs per day are allowed; worked minutes = sum of in→out intervals; breaks are implicit gaps (plus fixed unpaid `break_minutes` on the shift).
- Missing check-out: nightly job marks the day `present` with worked minutes to shift end minus a company-configurable penalty of 0 by default, flags the record `needs_review = true`, and notifies the employee to file a correction.
- Employee self-editable profile fields fixed set: phone, personal email, address, photo, emergency contacts. Everything else HR-only.
- Half-day leave allowed only on single-day requests (first/second half).
- Leave balance check uses the balance of the year containing the leave start date.
- Delete guards: departments/designations/shifts/leave types/locations cannot be soft-deleted while referenced by active employees or open records — 409 CONFLICT.

HRMS Blueprint — Talent & Money Modules (9–16)

> Document 02 of 11. Covers modules: 9 Payroll, 10 Employee Documents, 11 Recruitment, 12 Onboarding, 13 Performance Management, 14 Training/LMS, 15 Expenses & Reimbursements, 16 Employee Self-Service. Data shapes: `04-database.md` . Endpoints: `08-api.md` . Screens: `06-ui-ux.md` . Workflows: `07-workflows-and-automation.md` .

Module 9 — Payroll (Phase 2)

Purpose

Compute pay from locked attendance/leave data, run an approval gate, and publish immutable payslips. Country-agnostic core: statutory items are configuration, not code.

Features

- **Salary components** (`salary_components`): name, code, kind `earning|deduction` , calc type `fixed|percentage` (percentage of a named base component, typically BASIC), taxable flag, statutory flag, display order (P2). Examples seeded: BASIC, HRA (50% of BASIC), Special Allowance, PF (12% of BASIC, statutory), Professional Tax (fixed, statutory), TDS (manual override field).
- **Salary structures** (`salary_structures` + `salary_structure_components`): reusable templates listing components with default values/percentages (P2)
- **Employee salary assignment** (`employee_salaries`): structure + resolved per-component amounts + annual CTC, `effective_from` / `effective_to` — full revision history, exactly one active row per employee per date (P2)
- **Payroll runs** (`payroll_runs`): one per company per month; lifecycle `draft → processing → pending_approval → approved → paid` (P2)
- Inputs snapshot at `processing` : per employee — period days, payable days, LOP days (unapproved absence + unpaid leave), overtime approved minutes, one-off adjustments (bonus/advance/recovery) (P2)
- Calculation (canonical): payable_days = period_days − LOP_days; each component prorated by payable_days/period_days (statutory percentage components recompute on prorated base); gross = Σ earnings; net = gross − Σ deductions. Money in integer minor units. (P2)
- **Payslips** (`payslips` + `payslip_items`): generated at approval, status `draft → published → paid` ; PDF via worker; employee notified on publish (P2)
- Off-cycle adjustments and final settlement runs (offboarding handoff) (P2)
- Statutory country pack (configurable PF/ESI/PT/TDS presets for India, etc.) (P3)

User roles involved

- `hr_admin` : full module (`payroll.manage` , `payroll.approve` — approval by a second HR user recommended, enforced when >1 exists).
- `employee` : own payslips only (`payroll.view_own`).
- `manager` : **no access** (salary is never team-visible).

Main screens

`/hr/payroll` (runs list, run wizard: Prepare → Review lines → Approve → Publish; components & structures settings; employee salary tab on employee detail), `/me/payslips` .

Main actions

manage-components, manage-structures, assign-salary (with effective_from), create-run, process-run, edit-line-adjustments (draft only), submit-for-approval, approve, publish-payslips, mark-paid, download-payslip.

Approval workflows

Run: `draft` (editable inputs) → `processing` (system computing; locks inputs) → `pending_approval` (review screen; can revert to draft) → `approved` (payslips generated; irreversible) → `paid`. Requires attendance month lock first (422 otherwise). Approver must hold `payroll.approve`.

Required database entities

`salary_components`, `salary_structures`, `salary_structure_components`, `employee_salaries`, `payroll_runs`, `payslips`, `payslip_items`, `attendance_records` (read), `leave_requests` (read), `employees`, `employee_bank_accounts`.

Dependencies

Attendance (locked months), Leave (unpaid leave), Employee Management (bank accounts, status), Offboarding (settlement), Notifications, Documents (payslip PDFs in storage).

Module 10 — Employee Documents (Phase 2)

Purpose

Central, permission-controlled store of employee and company documents with expiry tracking.

Features

- Categories (`document_categories`): ID proof, contract, certificate, policy, payslip, other; per-category flags: employee-uploadable, expiry-required (P2)
- Upload via presigned URL; metadata row in `documents` (owner employee or company-level, category, name, file key, mime, size, expiry_date nullable, status `active|expired|archived`) (P2)
- Download via short-lived presigned URL after server permission check (P2)
- Expiry automation: daily scan flips status to `expired`, notifies employee + HR at T-30/T-7/T-0 (P2)
- Company documents (policies/handbook) visible to all employees (P2)
- Versioning-by-replacement (old file archived) (P2); e-sign (P3)

User roles involved

- `employee` : view own + company docs; upload into employee-uploadable categories.
- `manager` : view direct reports' **work** documents (category-flagged), not identity/salary docs.
- `hr_admin` : full CRUD, categories, exports.

Main screens

`/hr/documents` (all docs, expiring-soon view), `/me/documents`, Documents tab on employee detail.

Main actions

upload, download, replace, archive, set-expiry, manage-categories.

Approval workflows

None (uploads are direct). Optional HR verification flag per document (`verified_by`, `verified_at`) — informational, P2.

Required database entities

`documents`, `document_categories`, `employees`, `notifications`.

Dependencies

Employee Management; storage service; Notifications (expiry). Payroll writes payslip PDFs as documents (category `payslip`, system-managed).

Module 11 — Recruitment (Phase 2)

Purpose

Hiring pipeline from job posting to accepted offer, ending in conversion to an employee record.

Features

- Job postings: title, department, designation, location, openings, employment type, description, salary range (internal-only), status `draft|open|on_hold|closed` (P2)
- Candidates: name, email, phone, resume file, source (`referral|portal|agency|direct`), talent-pool reuse across jobs (P2)
- Applications: candidate × job posting; stage `applied → screening → interview → offer → hired | rejected` (rejection allowed from any stage, with reason) (P2)
- Interviews: per application, round name, scheduled_at, interviewer (employee), mode, feedback text, rating 1–5, recommendation `strong_yes|yes|no|strong_no` (P2)
- Offers: application, designation, proposed CTC (minor units), joining date, expiry date, status `draft|sent|accepted|declined|withdrawn` ; offer letter PDF (P2)
- Convert-to-employee: accepted offer → creates `employees` row (status `onboarding`), copies data, links back (P2)
- Public careers page + application form (P3)

User roles involved

- `hr_admin` : full pipeline management.
- `manager` (as interviewer): sees assigned interviews, submits feedback.
- `employee` : none (referrals P3).

Main screens

`/hr/recruitment` (jobs list; per-job kanban by stage; candidate drawer; interviews calendar; offers list).

Main actions

create-job, publish/close-job, add-candidate, move-stage, schedule-interview, submit-feedback, create-offer, send-offer, record-accept/decline, convert-to-employee.

Approval workflows

Offer requires `recruitment.approve` before `sent` (senior HR gate). Stage transitions are permission-gated (`recruitment.edit`) but not approval flows.

Required database entities

`job_postings` , `candidates` , `applications` , `interviews` , `offers` , `departments` , `designations` , `locations` , `employees` (interviewers; conversion target).

Dependencies

Departments & Designations, Employee Management (conversion), Onboarding (triggered post-conversion), Documents (resume/offer files), Notifications.

Module 12 — Onboarding (Phase 2)

Purpose

Turn a new hire into a productive, fully-set-up employee via reusable task checklists.

Features

- Templates (`onboarding_templates` + `onboarding_template_tasks`): named checklists; tasks have title, description, assignee role (`hr|it|manager|new_hire`), due offset days from join date, required flag (P2)
- Instantiation: on employee creation (status `onboarding`) HR picks a template → concrete `onboarding_tasks` rows with resolved assignees + due dates (P2)
- Task completion tracking with per-task status `pending|completed|skipped` , completed_by/at (P2)
- Completion gate: when all required tasks complete, HR activates employee (status → `active`) (P2)
- Progress view per new hire + overdue task nudges (P2)

User roles involved

- `hr_admin` : templates, instantiate, monitor, activate.
- `manager` / `employee` (new hire) / IT (an hr-designated user): complete assigned tasks.

Main screens

Onboarding tab within `/hr/employees/[id]` ; "My tasks" widget on `/dashboard` ; template settings under `/hr/employees` settings.

Main actions

manage-templates, start-onboarding, complete-task, skip-task (with reason), activate-employee.

Approval workflows

None per task; activation is the implicit gate (requires all required tasks complete, else 422).

Required database entities

`onboarding_templates` , `onboarding_template_tasks` , `onboarding_tasks` , `employees` , `notifications` .

Dependencies

Employee Management (status), Recruitment (conversion entry point), Documents (doc-collection tasks link), Notifications.

Module 13 — Performance Management (Phase 2)

Purpose

Lightweight goal-setting and review cycles: self review + manager review with ratings, per cycle.

Features

- Cycles (`performance_cycles`): name (e.g., "H1 2027"), period start/end, review deadline, status `draft|active|review|closed` (P2)
- Goals (`goals`): employee, optional cycle, title, description, weight %, progress %, status `not_started|in_progress|completed|cancelled` ; manager approval of goal set (P2)
- Reviews (`performance_reviews`): employee × cycle; self_rating + self_comments, manager_rating + manager_comments, final_rating; status `pending_self|pending_manager|completed` (P2)
- Rating scale 1–5 (company-configurable labels) (P2)
- Reminders before review deadline (P2)
- 360° feedback, calibration (P3)

User roles involved

- `employee` : own goals, self review.
- `manager` : approve goals, manager reviews for direct reports.
- `hr_admin` : cycles, monitor completion, final ratings export.

Main screens

`/me/performance` (goals + my reviews), `/team/performance` (reports' goals/reviews), `/hr/performance` (cycles admin, completion matrix).

Main actions

create-cycle, activate-cycle, add-goal, approve-goals, update-progress, submit-self-review, submit-manager-review, close-cycle, export-ratings.

Approval workflows

Goal set: employee proposes → manager approves (edits allowed before approval). Review chain: `pending_self` → employee submits → `pending_manager` → manager submits → `completed`. HR can reopen a review before cycle close.

Required database entities

`performance_cycles`, `performance_reviews`, `goals`, `employees`, `notifications`.

Dependencies

Employee Management (manager chain), Notifications, Reports (ratings distribution).

Module 14 — Training / LMS (Phase 3)

Purpose

Internal course delivery: content, enrollment, progress, certification.

Features

- Courses (`courses`): title, description, category, mandatory flag, deadline days (for mandatory), status `draft|published|archived` (P3)
- Lessons (`lessons`): ordered content units — rich text, video URL, file attachment; duration minutes (P3)
- Enrollments (`training_enrollments`): employee × course; status `enrolled|in_progress|completed` ; progress % (lessons completed / total); due_date for mandatory; score nullable (P3)
- Self-enroll (open courses) and HR/manager assignment (mandatory) (P3)
- Certificates (`certificates`): generated PDF on completion, verifiable id (P3)
- Quizzes/assessments with pass mark (P3, after core LMS)

User roles involved

- `employee` : browse, enroll, learn, download certificates.
- `manager` : view team progress, assign courses to reports.
- `hr_admin` : manage courses/lessons, assign, completion reports.

Main screens

`/me/training` (my courses, player view), `/hr/training` (courses admin, enrollment matrix).

Main actions

create-course, add-lessons, publish, enroll/assign, mark-lesson-complete, complete-course, issue-certificate, export-completion.

Approval workflows

None. Mandatory-course overdue drives escalating reminders.

Required database entities

`courses`, `lessons`, `training_enrollments`, `certificates`, `employees`, `notifications`.

Dependencies

Employee Management, Notifications, storage (videos/files, certificate PDFs).

Module 15 — Expenses & Reimbursements (Phase 2)

Purpose

Employee expense claims with receipts, manager approval, and tracked payout.

Features

- Categories (`expense_categories`): name, per-claim limit (minor units, nullable), receipt-required flag (P2)
- Expenses (`expenses`): employee, category, amount (minor units), expense date, description, receipt file(s); status `draft → submitted → approved | rejected → reimbursed` (P2)
- Approval by manager; HR override; auto-flag over-limit claims (P2)
- Reimbursements (`reimbursements`): payout record — expense(s) grouped, method `payroll|bank_transfer|cash` , paid_at, reference; marking paid flips expenses to `reimbursed` (P2)
- Payroll integration: approved expenses can attach to next payroll run as an earning adjustment (P2)

User roles involved

- `employee` : create/edit drafts, submit, view history.
- `manager` : approve/reject reports' claims.
- `hr_admin` : override, mark reimbursed, categories, export.

Main screens

`/me/expenses` , approvals inside `/team` (pending approvals widget), HR expenses view under `/hr/reports` area (expenses report) + admin list.

Main actions

create-draft, submit, approve, reject (reason required), mark-reimbursed, manage-categories, export.

Approval workflows

`draft` → employee submits → `submitted` → manager approve → `approved` → HR reimburses → `reimbursed` ; reject → `rejected` (employee may duplicate-and-resubmit). Approver ≠ claimant; over-limit requires `hr_admin` approval regardless of manager approval.

Required database entities

`expense_categories` , `expenses` , `reimbursements` , `employees` , `notifications` .

Dependencies

Employee Management, Notifications, Payroll (optional payout path), storage (receipts).

Module 16 — Employee Self-Service (Phase 1)

Purpose

The employee-facing surface: everything an employee can see/do about themselves, in one place. Not a data-owning module — a composition of own-tier permissions over other modules.

Features (what an employee can do, by module)

Area	Employee can (own scope)	Phase
Profile	view full profile; edit phone, personal email, address, photo, emergency contacts	P1

Area	Employee can (own scope)	Phase
Attendance	check-in/out; month calendar; raise corrections	P1
Leave	balances; apply; cancel; history	P1
Holidays	view calendar	P1
Documents	view own + company docs; upload allowed categories	P2
Payslips	view/download own	P2
Expenses	create, submit, track	P2
Performance	goals, self reviews	P2
Training	enroll, learn, certificates	P3
Notifications	view, mark read	P1
Offboarding	submit resignation, track exit tasks	P2
Settings	change password	P1

User roles involved

`employee` (primary); every user gets ESS regardless of extra roles.

Main screens

`/dashboard` (widgets: check-in card, leave balances, upcoming holidays, my pending tasks, announcements), all `/me/*` routes.

Main actions

See table above; all actions delegate to owning modules' services/APIs.

Approval workflows

Inherited from owning modules (leave, corrections, expenses, resignation).

Required database entities

None owned; reads/writes through other modules' services.

Dependencies

All Phase-relevant modules; Notifications.

Decisions made in this document

- Payroll approval: `payroll.approve` should be held by a different user than the run creator when the company has >1 HR user (four-eyes); enforced as a soft rule (warning) in P2, hard setting in P3.
- Percentage salary components resolve against a single named base component (BASIC) to avoid dependency graphs in v1; chained percentages are out of scope.
- Payslip PDFs are stored as `documents` rows (category `payslip`, system-managed, employee-visible, not employee-uploadable).
- Candidate PII kept in `candidates` distinct from `employees`; conversion copies data (no shared row), links via `employees.candidate_id` (nullable).
- Expense multi-receipt support = multiple `documents`-style file keys on the expense row (JSON array of storage keys) rather than a join table, to keep P2 lean.
- Rating scale fixed at 1–5 integers; labels configurable in system settings.

HRMS Blueprint — Platform Modules (17–23) & Reports Catalog

> Document 03 of 11. Covers modules: 17 Manager Dashboard, 18 HR Dashboard, 19 Notifications, 20 Reports & Analytics, 21 Offboarding, 22 Audit Logs, 23 System Settings. Data shapes: `04-database.md` . Endpoints: `08-api.md` . Screens: `06-ui-ux.md` . Workflows: `07-workflows-and-automation.md` .

Module 17 — Manager Dashboard (Phase 1)

Purpose

One screen where a manager sees team status and clears pending approvals. Scope is always direct reports (`employees.manager_id`).

Features

- Team presence today: present / absent / on leave / late counts + per-person list (P1)
- Pending approvals inbox: leave requests, attendance corrections (P1); expenses, resignations (P2)
- Team leave calendar (this week/month) (P1)
- Team headcount card and upcoming birthdays/anniversaries of reports (P1)
- Team performance summary (review completion, goal progress) (P2)
- Quick links to team reports (P2)

User roles involved

`manager` (T-scope); `hr_admin` may view any manager's team view (support/debug).

Main screens

`/team` (dashboard), plus `/team/attendance` , `/team/leave-approvals` , `/team/performance` , `/team/reports` .

Main actions

approve/reject from inbox (inline), navigate to detail views.

Approval workflows

Surfaces approvals owned by Leave / Attendance / Expenses / Offboarding modules; no own flows.

Required database entities

None owned — reads via module services (`attendance_records` , `leave_requests` , `attendance_corrections` , `expenses` , `employees` , `performance_reviews`).

Dependencies

Attendance, Leave, Employee Management; P2: Expenses, Performance, Offboarding.

Module 18 — HR Dashboard (Phase 1)

Purpose

Company-wide operational snapshot for HR: who's here, what needs action, what's trending.

Features / composition

KPI cards (P1): total active headcount; present today; on leave today; absent today; late today; pending approvals (all types); employees on probation; on notice (P2). **Charts:** headcount by department (bar) P1; attendance trend last 30 days (line: present%/late%) P1; leave usage by type this month (donut) P1; headcount trend 12 months (line) P2; attrition rate (line) P2; payroll cost trend (line, `payroll.view_all` only) P2. **Lists (P1):** today's absences without leave; pending leave approvals (oldest first); upcoming holidays; recent joiners; upcoming probation ends. **Quick actions (P1):** add employee, apply leave on behalf, add holiday, lock attendance month (P1); run payroll (P2).

User roles involved

`hr_admin` (A-scope). `super_admin` sees per-company selector (P3 multi-tenant).

Main screens

`/hr` (dashboard). Drill-through into each module's HR screen.

Main actions

Navigation + quick actions; no own mutations.

Approval workflows

None owned.

Required database entities

None owned — aggregates via module services.

Dependencies

All Phase-1 modules; P2 additions read Payroll/Offboarding/Performance.

Module 19 — Notifications (Phase 1)

Purpose

Deliver system events to users in-app and by email, from one dispatch service with templated messages. Full event catalog: `07-workflows-and-automation.md` §3.

Features

- `notifications` table rows per recipient: type, title, body, link (deep link into app), read_at (P1)
- Bell dropdown (latest 10, unread badge) + `/me/notifications` center with filters (P1)
- Mark read / mark all read (P1)
- Email fanout via worker queue (template registry; per-event enable toggle in settings) (P1)
- Announcements: HR-authored broadcast (`announcements` , rich text sanitized) with audience company-wide or department; `announcement_reads` receipts (P1)
- Daily digest option (P2); web push via PWA (P3); per-user channel preferences (P3)

User roles involved

All users receive; `hr_admin` creates announcements (`notifications.manage` for templates/toggles).

Main screens

Bell dropdown (global shell), `/me/notifications` , announcements composer in `/hr` area.

Main actions

mark-read, mark-all-read, create/publish announcement.

Approval workflows

None. Announcements publish directly (require `announcements.create`).

Required database entities

`notifications`, `announcements`, `announcement_reads`, `users`.

Dependencies

Every event-producing module; worker/queue; email service.

Module 20 — Reports & Analytics (Phase 1 basic lists; Phase 2 full catalog + exports; Phase 3 analytics)

Purpose

Answer operational and compliance questions with filterable, exportable tabular reports and dashboard KPIs.

Global report behavior

All reports: filter bar (date range presets + custom, department, location, employee, status where applicable), server-side pagination, column sort, CSV/XLSX export (P2; PDF where noted), permission = `reports.view_all` scoped by module-specific export permission for downloads. Manager variants under `/team/reports` auto-scope to direct reports with `reports.view_team`.

Report catalog

#	Report	Purpose	Key columns	Extra filters / grouping	KPIs on top	Phase
R1	Headcount	Current workforce composition	employee, code, dept, designation, location, type, status, join date	group by dept/location/type/status	total, by-status counts	P1
R2	Attendance summary	Per-employee month grid	employee, days present/absent/half/leave/holiday/week-off, worked hrs, OT hrs	month picker; dept	avg presence %	P1
R3	Absence report	Unapproved absences	employee, date, expected shift, reason (none)	date range, dept	absence count, top absentees	P1
R4	Late arrivals	Punctuality	employee, date, shift start, check-in, late minutes	threshold minutes; dept	late %, avg late mins	P1
R5	Overtime	OT liability	employee, date, OT minutes, approved?, approver	approved-only toggle	total OT hrs	P2
R6	Leave usage	Balances vs usage	employee, type, opening, accrued, used, carried, current	year, type, dept	utilization %	P1
R7	Leave calendar	Who's off when	date, employee, type, half-day	month, dept	days off this month	P1
R8	Payroll register	Run line items	employee, gross, per-component columns, deductions, net	run picker	total gross/net, employer cost	P2
R9	Payroll cost trend	Cost over time	month, gross, net, headcount, cost/head	12/24 months	YoY delta	P2
R10	Recruitment funnel	Pipeline health	job, applied, screening, interview, offer, hired, rejected, days-to-hire	job, date range	conversion %, avg time-to-hire	P2
R11	Turnover / attrition	Exits analysis	employee, join date, exit date, tenure, dept, reason	period, dept	attrition % (annualized), avg tenure	P2
R12	Performance distribution	Ratings spread	cycle, rating buckets 1–5 counts, dept breakdown	cycle, dept	avg rating, completion %	P2

#	Report	Purpose	Key columns	Extra filters / grouping	KPIs on top	Phase
R13	Training completion	Compliance	course, enrolled, in progress, completed, overdue	course, dept, mandatory-only	completion %	P3
R14	Expense summary	Spend control	employee, category, count, amount, status	period, category, dept	total spend, avg claim	P2
R15	Document expiry	Compliance	employee, document, category, expiry date, days left	category, window (30/60/90)	expiring ≤30d count	P2
R16	Audit activity	Sensitive-action review	timestamp, actor, action, entity, summary	actor, entity type, action, date	— (list only)	P1

Export events themselves are audit-logged (who exported what, when).

User roles involved

`hr_admin` (all, company scope); `manager` (R1–R7 team-scoped variants); `employee` none.

Main screens

`/hr/reports` (catalog grid → report page), `/team/reports`.

Main actions

run report, change filters, export, schedule email report (P3).

Approval workflows

None.

Required database entities

None owned; read-only aggregation over module tables. Heavy reports use read-optimized SQL (raw queries) not ORM loops.

Dependencies

Every data module; worker for async export generation of large files.

Module 21 — Offboarding (Phase 2)

Purpose

Controlled exit: resignation → approval → notice period → exit checklist → clearance → settlement handoff → deactivation. Ensures nothing (assets, access, money) is left dangling.

Features

- Resignation submission by employee (or HR on behalf): reason, requested last working day (P2)
- Manager approval → HR confirmation; notice period from employment terms (company setting default, overridable per employee); system computes last_working_day (P2)
- Employee status → `on_notice` on approval (P2)
- Exit checklist from template: asset return, access revocation, knowledge transfer, exit interview — tasks assigned to HR/IT/manager/employee with due dates (P2)
- Asset clearance flag per task; blocking: settlement cannot start until required tasks complete (P2)
- Final settlement inputs: unused paid leave encashment (days × per-day rate), pending expense reimbursements, recovery items, notice shortfall — handed to Payroll as a settlement run line (P2)
- Account deactivation on completion: `users.status → disabled`, sessions invalidated, employee `exited`, personal data retention clock starts (P2)
- Statuses (`offboarding_requests.status`): `initiated → in_progress → cleared → settled → completed` (P2)

User roles involved

- `employee` : submit resignation, complete own tasks, view exit status.
- `manager` : approve resignation of direct reports, complete assigned tasks.
- `hr_admin` : confirm, manage checklist, settlement, deactivate.

Main screens

Offboarding tab in `/hr/employees/[id]` ; resignation dialog in `/me/profile` ; exit-pipeline list view in `/hr/employees` (filter: on_notice).

Main actions

submit-resignation, approve-resignation, set-last-working-day, start-checklist, complete-task, mark-cleared, record-settlement, complete-offboarding.

Approval workflows

Resignation: employee submits (`initiated`) → manager approves → HR confirms (sets last_working_day, status `in_progress`). Withdrawal allowed while `initiated` / `in_progress` before last working day (HR approves withdrawal; status back to active employment, request `cancelled` — modeled as terminal note, request row kept). `cleared` requires all required tasks done; `settled` requires settlement recorded; `completed` performs deactivation.

Required database entities

`offboarding_requests` , `offboarding_tasks` , `onboarding_templates` (shared template mechanism — offboarding templates flagged by `kind`), `employees` , `users` , `leave_balances` (encashment read), `expenses` (pending read), `payroll_runs` (settlement), `notifications` , `audit_logs` .

Dependencies

Employee Management, Leave (encashment), Expenses, Payroll (settlement), Authentication (deactivation), Notifications.

Module 22 — Audit Logs (Phase 1)

Purpose

Append-only, tamper-evident record of every sensitive action for accountability and compliance.

What is logged (catalog)

- Auth: login success/failure, lockout, password change/reset, invite issued/accepted, user disabled
- RBAC: role assigned/removed, permission set changed
- Employee: create, sensitive-field edit (name/DOB/IDs/bank), status change, delete
- Attendance: manual entry, correction approve/reject, month lock/unlock
- Leave: approve/reject/cancel, balance adjustment, policy change
- Payroll: component/structure change, salary assignment, run state transitions, payslip publish (P2)
- Documents: upload, download of sensitive categories, delete (P2)
- Recruitment: offer sent/accepted (P2)
- Offboarding: every state transition (P2)
- Settings: any change
- Exports: any report/data export (who, what, filter params)

Log record shape

`audit_logs` : id, company_id, actor_user_id (nullable for system jobs), action (verb string e.g. `leave.approve`), entity_type, entity_id, before (jsonb, nullable), after (jsonb, nullable), ip, user_agent, created_at. **Append-only**: no update/delete API; DB role used by the app has no UPDATE/DELETE grant on this table; RLS still scopes reads by company.

Features

- Automatic emission from service layer via domain events (P1)
- Viewer with filters: date range, actor, action, entity type, entity id (P1)
- Detail drawer showing before/after diff (P1)
- Retention job (default 2 years, configurable) (P2)
- Export (P1, itself logged)

User roles involved

`hr_admin` (company scope read), `super_admin` (all). No write API for anyone.

Main screens

`/admin/audit-logs` .

Main actions

filter, inspect diff, export.

Approval workflows

None.

Required database entities

`audit_logs` .

Dependencies

All modules emit; none consumed.

Module 23 — System Settings (Phase 1)

Purpose

Central configuration store (`system_settings` : key, value jsonb, company_id nullable → null = global/platform default; company row overrides global).

Settings catalog

Key	Scope	Type	Default	Phase
`general.timezone`	company	string (IANA)	Asia/Kolkata	P1
`general.currency`	company	char(3)	INR	P1
`general.date_format`	company	string	DD/MM/YYYY	P1
`general.working_days`	company	int[] weekdays	[1,2,3,4,5]	P1
`general.leave_year_start_month`	company	int 1–12	1	P1
`attendance.join_mid_month_cutoff_day`	company	int	15	P1
`attendance.missing_checkout_policy`	company	enum flag+penalty	flag_only	P1
`attendance.auto_absent_after_days`	company	int	1	P1
`leave.sandwich_rule_default`	company	bool	false	P1
`leave.allow_negative_balance`	company	bool	false	P1
`security.password_min_length`	global	int	10	P1
`security.lockout_threshold`	global	int	5	P1

Key	Scope	Type	Default	Phase
`security.session_hours`	global	int	12	P1
`notifications.email_enabled`	company	bool	true	P1
`notifications.event_toggles`	company	jsonb map	all on	P2
`payroll.pay_day`	company	int	1	P2
`payroll.notice_period_days_default`	company	int	30	P2
`payroll.leave_encashment_enabled`	company	bool	true	P2
`performance.rating_labels`	company	jsonb	1–5 labels	P2
`retention.*` (per entity)	global	jsonb	see 09-security	P2

User roles involved

`hr_admin` edits company-scope keys (`settings.manage` company); `super_admin` edits global keys.

Main screens

`/admin/settings` (grouped tabs: General, Attendance, Leave, Security, Notifications, Payroll P2).

Main actions

edit-setting (typed forms per group; every change audit-logged with before/after).

Approval workflows

None.

Required database entities

`system_settings` .

Dependencies

None; all modules read settings through a cached settings service (cache invalidated on write).

Decisions made in this document

- Offboarding checklists reuse the onboarding template mechanism via a `kind` discriminator (`onboarding|offboarding`) on `onboarding_templates` — one mechanism, two flows.
- Settings storage = key/value jsonb with global-then-company override resolution; typed accessors in code guard against shape drift.
- Report exports > 5,000 rows generate asynchronously in the worker and deliver via notification with download link; smaller exports stream synchronously.
- Audit `before` / `after` store only changed fields (diff), not whole rows, except state transitions which store `{status: from→to}` plus context ids.
- Manager dashboard birthday/anniversary widgets show day+month only (no birth year) — privacy default.

HRMS Blueprint — Database Schema & Multi-Tenant Model

> Document 04 of 11. **Canonical for all data shapes.** Where any other doc disagrees with this one on tables/columns/enums, this doc wins.

1. Conventions

- PostgreSQL 16, Prisma ORM, `prisma migrate`.
- Tables: snake_case plural. PK: `id uuid` (default `gen_random_uuid()`).
- **`company_id uuid NOT NULL`** on every tenant-scoped table (all tables below except `roles`, `permissions`, `role_permissions` global seeds — see notes — and global `system_settings` rows). Always indexed; member of scoped unique keys `(company_id, code)`.
- `created_at timestamptz NOT NULL DEFAULT now()`, `updated_at timestamptz NOT NULL` (Prisma `@updatedAt`) on **all** tables. Not repeated per table below.
- Soft delete: `deleted_at timestamptz NULL` on master/people data only (marked ⚑SD below). Transactional/immutable rows (attendance, payslips, audit, notifications) never soft-delete — status fields instead.
- Money: `bigint` integer minor units; currency lives on `companies.currency char(3)`.
- Dates: `date` for calendar days, `timestamptz` (UTC) for instants, `time` for shift times.
- Enum columns implemented as Postgres enums via Prisma; values are canonical (listed per table).
- FK on-delete default: `RESTRICT` (protect history); exceptions noted.

2. Schema

2.1 Org

companies — tenant root. ⚑SD

column	type	null	notes
id	uuid	no	PK
name	varchar(160)	no	display name
legal_name	varchar(200)	yes	
slug	varchar(60)	no	UNIQUE (platform-wide)
logo_key	varchar(255)	yes	storage key
address	text	yes	
contact_email	varchar(160)	yes	
timezone	varchar(64)	no	IANA, default 'Asia/Kolkata'
currency	char(3)	no	default 'INR'
status	enum	no	`active|suspended`
Indexes: `slug` unique. (No company_id — it *is* the tenant.)			

locations — offices/branches. ⚑SD id PK; company_id FK→companies; name varchar(120); code varchar(30); address text null; timezone varchar(64) null (falls back to company). Unique `(company_id, code)`. Index `(company_id)`.

departments — org units. ⚑SD id PK; company_id FK; name varchar(120); code varchar(30); head_employee_id uuid null FK→employees (SET NULL). Unique `(company_id, code)`. Index `(company_id)`.

designations — job titles. ⚑SD id PK; company_id FK; title varchar(120); code varchar(30); level int no default 1. Unique `(company_id, code)`. Index `(company_id, level)`.

system_settings — key/value config. id PK; company_id uuid **null** FK (null = global default); key varchar(120); value jsonb; updated_by uuid null FK→users. Unique `(company_id, key)` (Postgres: use `COALESCE` unique index or partial unique for null company_id). Index `(key)`.

2.2 Auth

users — login identity (not every employee must have one immediately). id PK; company_id FK; email varchar(160); password_hash varchar(255) null (null until invite accepted); status enum `invited|active|disabled`; session_version int no default 1 (bump = logout everywhere); failed_login_count int default 0; locked_until timestamptz null; last_login_at timestamptz null. Unique `(company_id, email)`; global unique index on `lower(email)` (login has no company context pre-auth). Index `(company_id, status)`.

roles — role definitions. Seeded per company (system roles) with `is_system=true`. id PK; company_id FK; name varchar(60) (`super_admin|hr_admin|manager|employee` + custom P3); description text null; is_system bool default false. Unique `(company_id, name)`.

permissions — global catalog (no company_id; platform-defined). id PK; code varchar(80) UNIQUE (`module.action`); module varchar(40); action varchar(20); description text null.

role_permissions — grant mapping. id PK; role_id FK→roles (CASCADE); permission_id FK→permissions (CASCADE). Unique `(role_id, permission_id)`. Tenant scoping inherited through role.

user_roles — user↔role. id PK; user_id FK→users (CASCADE); role_id FK→roles (CASCADE); assigned_by uuid null FK→users. Unique `(user_id, role_id)`.

password_reset_tokens — reset + invite tokens. id PK; user_id FK→users (CASCADE); token_hash varchar(255) (SHA-256 of token; raw never stored); kind enum `reset|invite`; expires_at timestamptz; used_at timestamptz null. Index `(user_id)`, `(token_hash)` unique.

2.3 People

employees — the master. ⚑SD

column	type	null	notes
id	uuid	no	PK
company_id	uuid	no	FK
user_id	uuid	yes	FK→users, UNIQUE (one employee per user)
candidate_id	uuid	yes	FK→candidates (recruitment origin)
employee_code	varchar(30)	no	auto-generated
first_name / last_name	varchar(80)	no/yes	
work_email	varchar(160)	yes	
personal_email	varchar(160)	yes	
phone	varchar(30)	yes	
date_of_birth	date	yes	
gender	enum	yes	`male|female|other|undisclosed`
address	text	yes	
photo_key	varchar(255)	yes	storage
department_id / designation_id / location_id	uuid	no	FKs (RESTRICT)
manager_id	uuid	yes	FK→employees self-ref (SET NULL)
employment_type	enum	no	`full_time|part_time|contract|intern`
status	enum	no	`onboarding|active|on_notice|exited`

column	type	null	notes
join_date	date	no	
probation_end_date	date	yes	
confirmation_date	date	yes	
exit_date	date	yes	
notice_period_days	int	yes	overrides company default
Unique `(company_id, employee_code)` ; Indexes `(company_id, status)` , `(company_id, department_id)` , `(manager_id)` , `(user_id)` .			

employee_bank_accounts — payout details (sensitive; column-encrypt account_number). ⚑SD id PK; company_id FK; employee_id FK→employees (CASCADE); account_holder varchar(120); account_number_enc bytea; ifsc_or_swift varchar(20); bank_name varchar(120); is_primary bool default true. Index `(employee_id)` . Partial unique `(employee_id) WHERE is_primary` .

emergency_contacts — ⚑SD id PK; company_id FK; employee_id FK (CASCADE); name varchar(120); relationship varchar(60); phone varchar(30); is_primary bool default false. Index `(employee_id)` .

2.4 Time

shifts — working-time rules. ⚑SD id PK; company_id FK; name varchar(80); code varchar(30); start_time time; end_time time (may be < start_time = overnight); grace_minutes int default 10; half_day_threshold_minutes int; break_minutes int default 0; week_off_days int[] (0=Sun…6=Sat) default `{0,6}` ; is_default bool default false. Unique `(company_id, code)` . Partial unique `(company_id) WHERE is_default` .

employee_shifts — assignment history. id PK; company_id FK; employee_id FK (CASCADE); shift_id FK→shifts; effective_from date; effective_to date null. Index `(employee_id, effective_from)` . App-enforced: no overlapping ranges per employee (exclusion constraint `EXCLUDE USING gist` optional hardening).

attendance_punches — immutable raw events. id PK; company_id FK; employee_id FK; punched_at timestamptz; direction enum `in|out` ; source enum `web|mobile|biometric|manual` ; note varchar(255) null; created_by uuid null FK→users (manual entries). Index `(company_id, employee_id, punched_at)` . No soft delete; corrections supersede.

attendance_records — one per employee per date (computed). id PK; company_id FK; employee_id FK; work_date date; status enum `present|absent|half_day|on_leave|holiday|week_off` ; source enum `web|mobile|biometric|manual` ; first_in timestamptz null; last_out timestamptz null; worked_minutes int default 0; late_minutes int default 0; overtime_minutes int default 0; overtime_approved bool default false; overtime_approved_by uuid null FK→users; needs_review bool default false; locked bool default false. Unique `(company_id, employee_id, work_date)` . Indexes `(company_id, work_date)` , `(company_id, work_date, status)` .

attendance_corrections — regularization requests. id PK; company_id FK; employee_id FK; work_date date; requested_in timestamptz null; requested_out timestamptz null; requested_status enum(attendance status) null; reason text; status enum `pending|approved|rejected|cancelled` ; reviewed_by uuid null FK→users; reviewed_at timestamptz null; review_note text null. Indexes `(company_id, status)` , `(employee_id, work_date)` .

attendance_month_locks *(additional)* — payroll-safe month freeze. id PK; company_id FK; year int; month int; locked_by FK→users; locked_at timestamptz. Unique `(company_id, year, month)` .

2.5 Leave

leave_types — ⚑SD id PK; company_id FK; name varchar(80); code varchar(20); is_paid bool; color varchar(9) null; requires_attachment bool default false. Unique `(company_id, code)` .

leave_policies — accrual rules per type. id PK; company_id FK; leave_type_id FK (CASCADE); accrual_frequency enum `monthly|yearly|none` ; accrual_amount numeric(5,2); max_carry_forward numeric(5,2) default 0; max_negative numeric(5,2) default 0; min_notice_days int default 0; max_consecutive_days int null; applicable_after_probation bool default false; sandwich_rule bool default false. Unique `(company_id, leave_type_id)` (one active policy per type; revisions overwrite + audit).

leave_balances — per employee/type/year. id PK; company_id FK; employee_id FK (CASCADE); leave_type_id FK; year int; opening numeric(5,2) default 0; accrued numeric(5,2) default 0; used numeric(5,2) default 0; carried_forward numeric(5,2) default 0; adjusted numeric(5,2) default 0 (manual HR +/-); current numeric(5,2) GENERATED/maintained =

opening+accrued+carried_forward+adjusted−used. Unique `(company_id, employee_id, leave_type_id, year)`. Index `(employee_id, year)`.

leave_requests id PK; company_id FK; employee_id FK; leave_type_id FK; start_date date; end_date date; half_day enum `none|first_half|second_half` default none; days numeric(4,2) (computed working days); reason text; attachment_key varchar(255) null; status enum `pending|approved|rejected|cancelled`; approver_id uuid null FK→employees (resolved manager at submit); reviewed_by uuid null FK→users; reviewed_at timestamptz null; review_note text null. Indexes `(company_id, status)`, `(employee_id, start_date)`, `(approver_id, status)`.

holidays id PK; company_id FK; location_id uuid null FK (null = company-wide); name varchar(120); holiday_date date; is_optional bool default false (P2). Unique `(company_id, location_id, holiday_date, name)`. Index `(company_id, holiday_date)`.

2.6 Payroll (Phase 2 tables, in schema from day 1 migrations set 2)

salary_components — ⚑SD id PK; company_id FK; name varchar(80); code varchar(30); kind enum `earning|deduction`; calc_type enum `fixed|percentage`; percent_of_component_id uuid null FK→salary_components (base, e.g. BASIC); default_value bigint null (minor units or basis points for percentage); taxable bool default true; statutory bool default false; display_order int default 0. Unique `(company_id, code)`.

salary_structures — templates. ⚑SD id PK; company_id FK; name varchar(80); description text null. Unique `(company_id, name)`.

salary_structure_components id PK; company_id FK; structure_id FK (CASCADE); component_id FK→salary_components; value bigint (minor units, or basis points if percentage). Unique `(structure_id, component_id)`.

employee_salaries — assignment + revision history. id PK; company_id FK; employee_id FK; structure_id uuid null FK; annual_ctc bigint; components jsonb (resolved `[ {component_id, code, kind, monthly_amount} ]` snapshot); effective_from date; effective_to date null; created_by FK→users. Index `(employee_id, effective_from)`. App-enforced non-overlap.

payroll_runs id PK; company_id FK; year int; month int; status enum `draft|processing|pending_approval|approved|paid`; period_days int; input_snapshot jsonb null (per-employee attendance/leave summary at processing); created_by FK→users; approved_by uuid null FK→users; approved_at timestamptz null; paid_at timestamptz null. Unique `(company_id, year, month)`. Index `(company_id, status)`.

payslips — immutable after publish. id PK; company_id FK; payroll_run_id FK (CASCADE while draft; RESTRICT conceptually after approve — enforced in app); employee_id FK; period_days int; payable_days numeric(4,1); lop_days numeric(4,1); gross bigint; total_deductions bigint; net bigint; status enum `draft|published|paid`; pdf_key varchar(255) null; published_at timestamptz null. Unique `(payroll_run_id, employee_id)`. Index `(employee_id)`, `(company_id, status)`.

payslip_items — line items. id PK; company_id FK; payslip_id FK (CASCADE); component_code varchar(30); component_name varchar(80); kind enum `earning|deduction`; amount bigint; display_order int. Index `(payslip_id)`.

2.7 Documents

document_categories — ⚑SD id PK; company_id FK; name varchar(80); code varchar(30); employee_uploadable bool default false; expiry_required bool default false; manager_visible bool default false; system_managed bool default false (e.g. payslip). Unique `(company_id, code)`.

documents — metadata; file in object storage. ⚑SD id PK; company_id FK; category_id FK; employee_id uuid null FK (null = company-level doc); name varchar(160); file_key varchar(255); mime_type varchar(120); size_bytes bigint; expiry_date date null; status enum `active|expired|archived`; uploaded_by FK→users; verified_by uuid null FK→users; verified_at timestamptz null. Indexes `(company_id, category_id)`, `(employee_id)`, `(company_id, expiry_date) WHERE expiry_date IS NOT NULL`.

2.8 Recruitment (Phase 2)

job_postings — ⚑SD id PK; company_id FK; title varchar(160); department_id FK; designation_id FK; location_id FK; openings int default 1; employment_type enum(as employees); description text; salary_min / salary_max bigint null (internal); status enum `draft|open|on_hold|closed`; published_at timestamptz null. Index `(company_id, status)`.

candidates — ⚑SD id PK; company_id FK; first_name varchar(80); last_name varchar(80) null; email varchar(160); phone varchar(30) null; resume_key varchar(255) null; source enum `referral|portal|agency|direct`; notes text null. Unique `(company_id, email)`. Index `(company_id)`.

applications id PK; company_id FK; candidate_id FK (CASCADE); job_posting_id FK (CASCADE); stage enum `applied|screening|interview|offer|hired|rejected`; rejection_reason text null; stage_changed_at timestamptz. Unique

`(candidate_id, job_posting_id)`. Index `(company_id, job_posting_id, stage)`.

interviews id PK; company_id FK; application_id FK (CASCADE); round_name varchar(80); scheduled_at timestamptz; mode enum `onsite|video|phone`; interviewer_employee_id FK→employees; status enum `scheduled|completed|cancelled`; feedback text null; rating int null (1–5); recommendation enum `strong_yes|yes|no|strong_no` null. Index `(application_id)`, `(interviewer_employee_id, scheduled_at)`.

offers id PK; company_id FK; application_id FK UNIQUE; designation_id FK; annual_ctc bigint; joining_date date; expires_on date; status enum `draft|sent|accepted|declined|withdrawn`; letter_key varchar(255) null; approved_by uuid null FK→users; sent_at / responded_at timestamptz null. Index `(company_id, status)`.

2.9 Boarding (Phase 2)

onboarding_templates — shared by on/offboarding via `kind`. ⚑SD id PK; company_id FK; name varchar(120); kind enum `onboarding|offboarding`; description text null. Unique `(company_id, name, kind)`.

onboarding_template_tasks id PK; company_id FK; template_id FK (CASCADE); title varchar(160); description text null; assignee_kind enum `hr|it|manager|new_hire`; due_offset_days int default 0; required bool default true; sort_order int. Index `(template_id)`.

onboarding_tasks — instantiated checklist items. id PK; company_id FK; employee_id FK (CASCADE); template_task_id uuid null FK; title varchar(160); description text null; assignee_user_id FK→users; due_date date; required bool; status enum `pending|completed|skipped`; completed_by uuid null FK→users; completed_at timestamptz null; skip_reason text null. Indexes `(employee_id, status)`, `(assignee_user_id, status)`.

offboarding_requests id PK; company_id FK; employee_id FK UNIQUE-active (partial unique `WHERE status NOT IN ('completed')`); reason text; requested_last_day date; notice_period_days int; last_working_day date null; status enum `initiated|in_progress|cleared|settled|completed`; approved_by_manager uuid null FK→users; confirmed_by_hr uuid null FK→users; settlement jsonb null (leave encashment days/amount, recoveries, notice shortfall); completed_at timestamptz null. Index `(company_id, status)`.

offboarding_tasks — same shape as onboarding_tasks with offboarding_request_id FK (CASCADE) instead of employee-only link. id PK; company_id FK; offboarding_request_id FK; title; description; assignee_user_id FK; due_date; required bool; blocking bool default false (blocks `cleared`); status enum `pending|completed|skipped`; completed_by/at; skip_reason. Index `(offboarding_request_id, status)`.

2.10 Performance (Phase 2)

performance_cycles — ⚑SD id PK; company_id FK; name varchar(80); period_start date; period_end date; review_deadline date; status enum `draft|active|review|closed`. Unique `(company_id, name)`.

goals id PK; company_id FK; employee_id FK; cycle_id uuid null FK; title varchar(160); description text null; weight int default 0 (percent); progress int default 0 (0–100); status enum `not_started|in_progress|completed|cancelled`; approved_by uuid null FK→users. Index `(employee_id, cycle_id)`.

performance_reviews id PK; company_id FK; cycle_id FK; employee_id FK; self_rating int null; self_comments text null; self_submitted_at timestamptz null; manager_rating int null; manager_comments text null; manager_submitted_at timestamptz null; final_rating int null; status enum `pending_self|pending_manager|completed`. Unique `(cycle_id, employee_id)`. Index `(company_id, status)`.

2.11 Training (Phase 3)

courses — ⚑SD id PK; company_id FK; title varchar(160); description text; category varchar(60) null; mandatory bool default false; deadline_days int null; status enum `draft|published|archived`. Index `(company_id, status)`.

lessons id PK; company_id FK; course_id FK (CASCADE); title varchar(160); content_type enum `text|video|file`; body text null; media_key varchar(255) null; duration_minutes int null; sort_order int. Index `(course_id, sort_order)`.

training_enrollments id PK; company_id FK; course_id FK; employee_id FK; status enum `enrolled|in_progress|completed`; progress int default 0; due_date date null; completed_at timestamptz null; score int null; completed_lesson_ids uuid[] default '{}'. Unique `(course_id, employee_id)`. Index `(employee_id, status)`.

certificates id PK; company_id FK; enrollment_id FK UNIQUE; employee_id FK; course_id FK; certificate_no varchar(40) UNIQUE(platform); pdf_key varchar(255); issued_at timestamptz.

2.12 Expenses (Phase 2)

expense_categories — ⚑SD id PK; company_id FK; name varchar(80); code varchar(30); per_claim_limit bigint null; receipt_required bool default true. Unique `(company_id, code)`.

expenses id PK; company_id FK; employee_id FK; category_id FK; amount bigint; expense_date date; description text; receipt_keys jsonb default '[]' (array of storage keys); status enum `draft|submitted|approved|rejected|reimbursed`; reviewed_by uuid null FK→users; reviewed_at timestamptz null; review_note text null; reimbursement_id uuid null FK→reimbursements. Indexes `(company_id, status)`, `(employee_id, expense_date)`.

reimbursements id PK; company_id FK; employee_id FK; total_amount bigint; method enum `payroll|bank_transfer|cash`; reference varchar(120) null; paid_at timestamptz null; payroll_run_id uuid null FK; created_by FK→users. Index `(employee_id)`.

2.13 Comms

notifications — per-recipient rows; no soft delete. id PK; company_id FK; user_id FK (CASCADE); type varchar(60) (event key); title varchar(160); body text; link varchar(255) null; read_at timestamptz null. Indexes `(user_id, read_at)`, `(user_id, created_at DESC)`.

announcements — ⚑SD id PK; company_id FK; title varchar(160); body_html text (sanitized server-side); audience enum `company|department`; department_id uuid null FK; published_at timestamptz null; created_by FK→users. Index `(company_id, published_at DESC)`.

announcement_reads id PK; company_id FK; announcement_id FK (CASCADE); user_id FK (CASCADE); read_at timestamptz. Unique `(announcement_id, user_id)`.

2.14 Ops

audit_logs — append-only (app DB role: INSERT + SELECT only). id PK (uuid v7 preferred for time-ordering); company_id FK; actor_user_id uuid null FK (null = system job); action varchar(80) (`module.action` verb); entity_type varchar(60); entity_id uuid null; before jsonb null; after jsonb null; ip inet null; user_agent varchar(255) null; created_at timestamptz. Indexes `(company_id, created_at DESC)`, `(company_id, entity_type, entity_id)`, `(actor_user_id, created_at DESC)`. No updated_at (immutable).

---

3. Relationship narrative

- **Company → everything**: `companies` is the tenant root; `locations`, `departments`, `designations`, and every operational table carry `company_id`. Company → Departments → Employees is the primary org drill-down (department FK on employee; department optionally points back to a head employee).
- **User ↔ Employee**: `employees.user_id` (unique, nullable) — an employee may exist before being invited; a user always belongs to one company; roles attach to the user, not the employee.
- **Employee → manager**: self-referential `manager_id` defines the team tier used by all `view_team` permissions and approval routing.
- **Employee → time**: one `attendance_records` row per day (unique employee+date), derived from many `attendance_punches`; `attendance_corrections` reference the day and, when approved, trigger recomputation; `employee_shifts` supplies the shift rules effective on that date; `attendance_month_locks` freezes months.
- **Employee → leave**: `leave_balances` (per type/year) are mutated only by accrual jobs, approval/cancellation of `leave_requests`, and audited HR adjustments; requests resolve day counts against `holidays` + shift week-offs.
- **Employee → money**: `employee_salaries` (versioned by effective dates) feed `payroll_runs`, which emit one `payslips` row per employee with `payslip_items` lines; expenses may flow into a run via `reimbursements.payroll_run_id`.
- **Recruitment chain**: `candidates` × `job_postings` → `applications` (stage machine) → `interviews` (n per application) → `offers` (1 per application) → on accept, an `employees` row is created carrying `candidate_id` provenance → onboarding tasks instantiate.
- **Boarding chains**: `onboarding_templates` (+template tasks, kind-discriminated) instantiate into `onboarding_tasks` (join-date offsets) or `offboarding_tasks` (linked to an `offboarding_requests` row that drives the exit state machine and final settlement handoff).
- **Performance & training**: `performance_reviews` unique per (cycle, employee) with goals optionally cycle-linked; `training_enrollments` unique per (course, employee) with `certificates` 1:1 on completion.

- **Comms**: `notifications` are per-user fanout rows; `announcements` broadcast with `announcement_reads` receipts.
- **audit_logs** uses polymorphic `entity_type` + `entity_id` — no FK, by design (logs must survive entity deletion); `before/after` jsonb captures diffs.

---

4. Multi-tenant model

4.1 Strategy: shared database, shared schema, `company_id` column

Chosen over schema-per-tenant (migration fan-out pain) and DB-per-tenant (operational overkill pre-scale). Compare: column strategy needs rigorous scoping — delivered by two independent layers below. Migration cost stays O(1) per release regardless of tenant count.

4.2 Layer 1 — tenant-scoped Prisma client (app layer, primary)

A Prisma client extension wraps every model operation:

```
// pseudo-code
const tenantDb = (companyId) => prisma.$extends({
  query: { $allModels: {
    $allOperations({ model, operation, args, query }) {
      if (TENANT_MODELS.has(model)) {
        args.where = { AND: [args.where ?? {}, { company_id: companyId }] }
        if (operation.startsWith('create')) args.data.company_id = companyId
      }
      return query(args)
    },
  }},
})
```

- Request context resolves `companyId` from the authenticated session **server-side** (never from client input).
- The raw unscoped client is exported only as `adminDb` and is importable only by `super_admin` platform services (lint rule + code review gate).

4.3 Layer 2 — PostgreSQL RLS (defense-in-depth)

Every tenant table gets RLS enabled from the first migration; each request's transaction sets `SET LOCAL app.company_id = '<uuid>'` (and `app.is_super_admin`):

```
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON employees
  USING (company_id = current_setting('app.company_id')::uuid
         OR current_setting('app.is_super_admin', true) = 'on');

-- identical policies on leave_requests, payslips, etc.
ALTER TABLE payslips ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON payslips
  USING (company_id = current_setting('app.company_id')::uuid
         OR current_setting('app.is_super_admin', true) = 'on');

ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_read ON audit_logs FOR SELECT
  USING (company_id = current_setting('app.company_id')::uuid
         OR current_setting('app.is_super_admin', true) = 'on');
CREATE POLICY tenant_insert ON audit_logs FOR INSERT
  WITH CHECK (company_id = current_setting('app.company_id')::uuid);
```

- App connects as a non-superuser role without `BYPASSRLS`.

- If the app layer ever leaks a query without scoping, RLS returns zero rows instead of another tenant's data.
- `super_admin` bypass = explicit session flag set only by platform-service code paths.

4.4 Why both layers

RLS alone can't express "manager sees only direct reports" cheaply, and app scoping alone is one forgotten `where` away from a breach. App layer = business scoping + good errors; RLS = last-line isolation guarantee. Tenant-isolation tests (two seeded companies, assert zero cross-reads) run in CI against both layers (see `10-roadmap-testing-deployment.md`).

4.5 Access tiers on top of tenancy

Tier	Who	Query shape (inside tenant scope)
own	employee	`where employee_id = ctx.employeeId` (or `user_id = ctx.userId`)
team	manager (`*.view_team`)	`where employee.manager_id = ctx.employeeId`
company	HR (`*.view_all`)	no additional filter (tenant scope only)
platform	super_admin	`adminDb`, RLS bypass flag; every use audit-logged

Object-level guards layered in services: approver ≠ requester; manager approvals verify the target's `manager_id`; salary/bank fields stripped from any non-`payroll.*`/non-self serialization (field-level allowlists in DTO mappers).

---

5. Seed data

1. Platform: full `permissions` catalog (every module.action used in `08-api.md`).
2. One company (pilot) with timezone/currency defaults.
3. Four system roles for the company + `role_permissions` per the matrix in `00-overview-and-roles.md` §6.4.
4. Default shift (09:30–18:30, grace 10, half-day threshold 240 min, week-off Sat/Sun) marked `is_default`.
5. Leave types Casual/Sick/Earned/Unpaid with policies (e.g., CL 1/month, SL 0.5/month, EL 1.25/month carry-forward ≤ 30, LWP unpaid no accrual).
6. Default document categories (P2 migration), expense categories (P2), settings rows for every `system_settings` key with defaults.
7. One `super_admin` user + one `hr_admin` user (invited state; passwords set via invite links). Dev-only: sample departments/designations/employees fixture behind a `SEED_DEMO=true` flag.

---

6. Migration & data lifecycle

- `prisma migrate dev` locally; `prisma migrate deploy` in CI against staging → prod; no `db push` outside prototypes.
- RLS policies and the `EXCLUDE`/partial-unique constraints live in customized migration SQL (Prisma doesn't model them) — committed alongside.
- Destructive migrations (drop/retype) require a two-release expand-contract pattern and a verified backup (checklist in `10-roadmap-testing-deployment.md`).
- Soft-deleted master rows excluded by a default `deleted_at IS NULL` filter in the Prisma extension; hard purges only via retention jobs per `09-security.md` §13.
- Enum growth is additive-only within a phase; renames = new value + backfill + removal across two releases.

Decisions made in this document

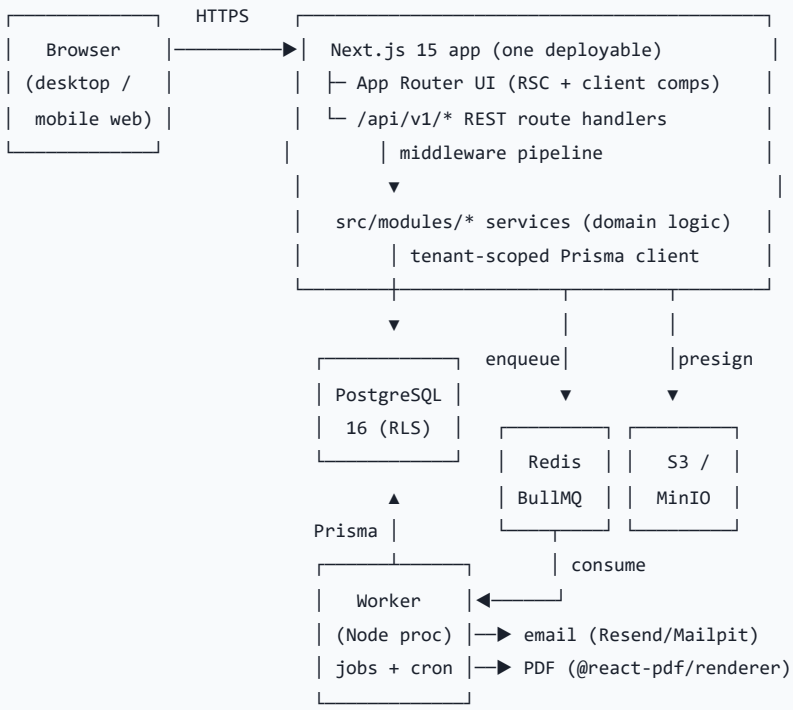
- `attendance_month_locks` added (additional table) — cleaner than a boolean scattered on records; `attendance_records.locked` mirrors it for query speed.
- `employee_salaries.components` stores a resolved jsonb snapshot so historical payslips never re-derive from mutated component definitions.

- `permissions` is a global (non-tenant) catalog; `roles` are per-company rows so Phase 3 custom roles need no schema change.
- Login uses a platform-wide unique `lower(email)` index because the login form has no company context; per-company email reuse is therefore not supported (acceptable: one identity per person per platform).
- uuid v7 recommended for `audit_logs` and other high-insert tables for index locality; standard v4 elsewhere.

HRMS Blueprint — Application Architecture

> Document 05 of 11. Stack decisions are canonical (see `00-overview-and-roles.md` §4.3). Data shapes: `04-database.md`.

1. Architecture overview



Modular monolith rule (canonical): all server domain logic lives in `src/modules/<module>/`; a module's tables are touched only by its own service functions; cross-module needs call the other module's exported service (e.g., payroll calls `attendanceService.getMonthSummary()`, never queries `attendance_records` itself). This keeps Payroll/Recruitment/LMS bolt-on and extraction-ready.

2. Frontend structure

- **Route groups**: `(auth)` = public auth pages with minimal layout; `(app)` = authenticated shell (topbar + permission-driven sidebar). Route paths are the canonical list in `00-overview-and-roles.md` /CORE routes: `/dashboard`, `/me/*`, `/team/*`, `/hr/*`, `/admin/*`.
- **Layouts**: `(app)/layout.tsx` loads session + permission set server-side; sidebar sections render from a nav config filtered by permissions (never by role name).
- **Server vs client components**: pages/server components fetch initial data (direct service calls, not HTTP, where co-located); interactive islands (tables, forms, dialogs, check-in button) are client components using TanStack Query against `/api/v1`.
- **Data fetching**: TanStack Query with query keys `[module, resource, params]`; mutations invalidate the affected keys; optimistic updates only for notifications mark-read and check-in button.
- **Forms**: react-hook-form + zodResolver, sharing the same zod schemas the API validates with (`src/modules/*/validators.ts` imported both sides).
- **Error/loading conventions**: per-route `loading.tsx` (skeletons) and `error.tsx` (retry boundary); mutation outcomes via toast; API error envelope mapped to field errors (`details`) or toast (`message`).

3. Backend/API structure

Every route handler is a thin adapter:

```
handler = withApi(
  { permission: 'leave.approve', schema: approveLeaveSchema },
  async (req, ctx) => leaveService.approve(ctx, req.valid)
)
```

`withApi` pipeline (order matters):

1. **authenticate** — verify JWT session cookie → `ctx.userId`, `ctx.companyId`, `ctx.employeeId`, `ctx.permissions`, `ctx.sessionVersion` check.
2. **tenant resolve** — build tenant-scoped Prisma client; open transaction sets `SET LOCAL app.company_id` (see `04-database.md` §4.3).
3. **requirePermission** — declared permission ∈ `ctx.permissions`, else 403.
4. **zod validate** — body/query → typed `req.valid`, else 400 with `details`.
5. **service call** — domain logic in `src/modules/*`; throws typed `AppError`s mapped to the envelope.

- **Transactions** live in services (`db.$transaction`) around multi-write operations (approve leave = update request + balance + notification row + audit row atomically).
- **Domain events**: services emit in-process events (`leave.approved`, `payslip.published`) via a typed emitter; handlers enqueue BullMQ jobs (email fanout, PDF generation) and write `audit_logs`. Events keep modules decoupled: payroll doesn't import the mailer.
- **Serialization**: DTO mappers per module with field allowlists (salary/bank fields only in payroll DTOs) — never `return prismaRow` directly.

---

4. Authentication flow

1. HR creates employee → "invite user" → `users` row (`invited`) + `password_reset_tokens` (kind `invite`, TTL 7d) → email with link `/reset-password?token=…&kind=invite`.
2. User sets password (argon2id) → status `active` → redirected to login.
3. Login (Auth.js credentials): verify hash, check `status='active'` and `locked_until`, issue JWT (claims: userId, companyId, sessionVersion) in httpOnly Secure SameSite=Lax cookie; sliding 12h expiry (`security.session_hours`).
4. Next.js middleware guards `(app)` routes → redirect to `/login`; API returns 401.
5. Logout clears cookie; "logout everywhere" bumps `users.session_version` (JWT claim mismatch → 401).
6. Forgot/reset: same token table, kind `reset`, TTL 1h, single-use, hashed at rest.

---

5. File storage

- Buckets private; one bucket, keys namespaced: `{companyId}/{module}/{entityId}/{uuid}-{sanitizedFilename}`.
- **Upload**: client asks `POST /api/v1/documents/presign-upload` (permission-checked, validates mime/size against category rules) → presigned PUT URL (TTL 10 min) → client uploads → client confirms → metadata row created.
- **Download**: `GET /api/v1/documents/:id/download` re-checks object-level permission → presigned GET URL (TTL 5 min) → redirect.
- Limits: 10 MB default, 50 MB for training video files (P3); allowlist mime types per category.
- Virus-scan hook point: post-upload confirm enqueues optional `scan-file` job (no-op P1/P2, ClamAV P3).

---

6. Notification service

- Single `notify(event, payload)` entry in `src/modules/notifications/service.ts`.

- Resolves recipients (event-specific resolver), writes `notifications` rows (in-app, instant), enqueues `send-email` jobs per recipient if `notifications.email_enabled` and event toggle on.
- **Template registry**: one file per event key (subject/body builders for email; title/body/link for in-app) — the catalog in `07-workflows-and-automation.md` §3 is the authoritative event list.
- Push channel (P3) plugs in as another fanout target; per-user preferences table (P3) filters channels before fanout.

---

7. Background jobs & scheduled tasks

Job	Trigger	Schedule (company TZ)	What it does	Phase
`send-email`	event	—	render template, send via mailer, retry ×5 backoff	P1
`attendance-daily-calc`	cron	00:30 daily	build/update yesterday's `attendance_records` from punches+shift+leave+holidays; mark absences; flag missing check-outs	P1
`late-arrival-flags`	within daily-calc	—	compute late_minutes; notify employee + manager per toggle	P1
`leave-accrual`	cron	1st of month 01:00	apply `leave_policies` accruals to `leave_balances`; prorate joiners	P1
`leave-year-rollover`	cron	leave-year start 02:00	carry-forward with caps; open new-year balances	P1
`birthday-anniversary`	cron	08:00 daily	notify team/HR per toggles	P1
`probation-end-reminder`	cron	08:00 daily	notify HR T-7 before `probation_end_date`	P1
`payslip-generate`	event (`payroll_run.approved`)	—	render payslip PDFs, store, publish, notify	P2
`document-expiry-scan`	cron	07:00 daily	T-30/T-7/T-0 notifications; flip `expired`	P2
`contract-expiry-scan`	within expiry scan	—	contract-category documents → HR notification	P2
`report-export`	event	—	large exports to file, notify with link	P2
`review-deadline-reminder`	cron	08:00 daily	performance review T-7/T-1 nudges	P2
`training-deadline-reminder`	cron	08:00 daily	mandatory course due nudges	P3
`daily-digest`	cron	18:00 daily	batched summary email (opt-in)	P2
`audit-retention`	cron	Sunday 03:00	purge per retention policy	P2
`scan-file`	event	—	virus scan hook	P3

Worker = separate Node process (`worker/index.ts`) sharing `src/modules` services via the same tenant-scoping utilities (jobs carry `companyId`; system actor for audit). Repeatable jobs registered idempotently on boot. Queue health (depth, failures) exposed at `/api/health` (see §9).

---

8. Integration seams (build the seam now, the integration later — P3)

- **Biometric devices**: `POST /api/v1/integrations/biometric/punches` (API-key auth per device, payload → `attendance_punches` with source `biometric`). Seam = punches table already sources-tagged.
- **Accounting**: payroll register export in a mapper interface (`toTallyCsv` , `toQuickBooksJson`). Seam = payslip/run DTOs.
- **Calendar**: leave/holiday iCal feed per user (signed URL). Seam = leave/holiday services.
- **Slack/Chat**: notification fanout target implementing the same channel interface as email.

---

9. Logging & error handling

- **pino** structured JSON logs; every request gets `request_id` (propagated to worker jobs); log line includes userId/companyId (never tokens/PII payloads).
- **Error taxonomy**: `AppError(code, httpStatus, message, details?)` subclasses — `ValidationError` 400, `AuthError` 401, `ForbiddenError` 403, `NotFoundError` 404, `ConflictError` 409, `BusinessRuleError` 422, `RateLimitError` 429. Route wrapper maps to the canonical envelope; unexpected errors log full stack, return generic 500 INTERNAL (no internals leaked).
- **Sentry** on web + worker (release-tagged); alert on new issue + job-failure spikes.
- `/api/health` : liveness (200) + readiness (DB ping, Redis ping, queue depth) for monitors.

---

10. Folder structure (full)

```
hrms/
├ .github/workflows/ci.yml           # lint, typecheck, test, build, migrate, deploy
├ docker-compose.yml                 # web, worker, postgres, redis, minio, mailpit
├ Dockerfile                         # multi-stage: web + worker targets
├ .env.example                       # every env var documented (10-roadmap §4)
├ prisma/
│  ├ schema.prisma
│  ├ migrations/                     # includes hand-written RLS/constraint SQL
│  └ seed.ts
├ src/
│  ├ app/
│  │  ├ (auth)/login/page.tsx
│  │  ├ (auth)/forgot-password/page.tsx
│  │  ├ (auth)/reset-password/page.tsx
│  │  ├ (app)/layout.tsx             # shell: topbar, sidebar (permission-driven)
│  │  ├ (app)/dashboard/page.tsx
│  │  ├
(app)/me/{profile,attendance,leave,documents,payslips,expenses,training,performance,notifications,settings}/page.tsx
│  │  ├ (app)/team/{page.tsx,attendance,leave-approvals,performance,reports}/
│  │  ├
(app)/hr/{page.tsx,employees,employees/[id],departments,attendance,leave,payroll,documents,recruitment,performance,training,
reports}/
│  │  ├ (app)/admin/{company,roles,locations,settings,audit-logs}/page.tsx
│  │  └ api/
│  │     ├ health/route.ts
│  │     └ v1/                       # one folder per resource; route.ts + [id]/route.ts + action routes
│  │        ├ auth/{login,logout,forgot-password,reset-password,accept-invite,me}/route.ts
│  │        ├ employees/…  departments/…  designations/…  locations/…
│  │        ├ shifts/…  employee-shifts/…
│  │        ├ attendance/{check-in,check-out,records,punches,corrections,locks}/…
│  │        ├ leave-types/…  leave-policies/…  leave-balances/…  leave-requests/…  holidays/…
│  │        ├ payroll/{components,structures,employee-salaries,runs,payslips}/…
│  │        ├ documents/…  document-categories/…
│  │        ├ job-postings/…  candidates/…  applications/…  interviews/…  offers/…
│  │        ├ onboarding/{templates,tasks}/…  offboarding/{requests,tasks}/…
│  │        ├ performance/{cycles,reviews,goals}/…  training/{courses,lessons,enrollments,certificates}/…
│  │        ├ expenses/…  expense-categories/…  reimbursements/…
│  │        ├ notifications/…  announcements/…  reports/…  audit-logs/…
│  │        └ roles/…  permissions/…  users/…  settings/…
│  ├ modules/                        # DOMAIN LOGIC — the heart
│  │  ├ auth/{service.ts,validators.ts,types.ts}
│  │  ├ employees/{service.ts,validators.ts,types.ts,dto.ts}
│  │  ├ org/{service.ts,validators.ts}          # company, locations, departments, designations
│  │  ├ attendance/{service.ts,calc.ts,validators.ts}   # calc.ts = pure day-computation (unit-tested hard)
│  │  ├ shifts/{service.ts,validators.ts}
│  │  ├ leave/{service.ts,accrual.ts,day-count.ts,validators.ts}
│  │  ├ holidays/{service.ts,validators.ts}
│  │  ├ payroll/{service.ts,engine.ts,validators.ts}    # engine.ts = pure calculation
│  │  ├ documents/{service.ts,validators.ts}
│  │  ├ recruitment/{service.ts,validators.ts}
│  │  ├ boarding/{service.ts,validators.ts}             # on/offboarding
│  │  ├ performance/{service.ts,validators.ts}
│  │  ├ training/{service.ts,validators.ts}
│  │  ├ expenses/{service.ts,validators.ts}
│  │  ├ notifications/{service.ts,templates/,resolvers.ts}
│  │  ├ reports/{service.ts,queries/}                   # raw SQL per report
│  │  ├ audit/{service.ts}
```

```
│   │   └─ settings/{service.ts,catalog.ts}
│   ├─ lib/
│   │   ├─ auth.ts            # Auth.js config, session helpers
│   │   ├─ db.ts              # prisma base + tenantDb() extension + adminDb
│   │   ├─ permissions.ts     # can(), requirePermission(), permission catalog types
│   │   ├─ api.ts             # withApi() pipeline, envelope, error mapping
│   │   ├─ errors.ts          # AppError taxonomy
│   │   ├─ storage.ts         # presign helpers
│   │   ├─ email.ts           # mailer interface (Resend/Mailpit impls)
│   │   ├─ queue.ts           # BullMQ queues/registrations
│   │   ├─ events.ts          # typed domain-event emitter
│   │   ├─ rate-limit.ts
│   │   └─ utils.ts
│   ├─ components/{ui/,shared/,charts/}   # shadcn primitives; DataTable, FilterBar, EmptyState, StatCard…
│   └─ hooks/                              # useSession, usePermissions, useDataTable…
├─ worker/
│   ├─ index.ts                          # boot, register processors + repeatables
│   └─ jobs/                             # one file per job in §7 table
├─ tests/
│   ├─ unit/                             # calc.ts, engine.ts, day-count.ts, accrual.ts
│   ├─ integration/                      # API+DB (testcontainers), permission matrix, tenant isolation
│   └─ e2e/                              # Playwright flows
└─ docs/                                 # this blueprint
```

Decisions made in this document

- Server components may call module services directly (same process) for initial page data; all mutations and client refetches go through `/api/v1` so permissions/validation have a single choke point either way (`withApi` logic reused by a `withPage` helper for RSC reads).
- Pure calculators (`attendance/calc.ts`, `payroll/engine.ts`, `leave/day-count.ts`, `leave/accrual.ts`) take plain inputs and return plain outputs — no DB access — so the hardest HR math is trivially unit-testable.
- Worker imports module services (shared code) rather than duplicating logic; jobs always carry `companyId` and run under a system actor id for audit.
- One S3 bucket with company-prefixed keys (simpler ops); per-tenant buckets deferred until compliance demands.

HRMS Blueprint — UI/UX Screen Specifications

> Document 06 of 11. Routes are canonical. Components: Tailwind CSS + shadcn/ui + lucide-react + Recharts.

1. Design principles & system

- **Look**: clean professional SaaS — white/neutral surfaces, one primary brand color, generous whitespace, 8px spacing grid, `Inter` (or system) typeface, subtle borders over shadows.
- **Layout anatomy** (authenticated shell): top bar (global search P2, notifications bell, profile menu) + left sidebar (permission-driven sections, collapsible) + content area (page header: title, breadcrumb, primary action button; then content).
- **Responsive strategy**: desktop-first; ≤ md breakpoints: sidebar → bottom nav (Employee role) or hamburger drawer; data tables collapse to stacked cards showing key columns; primary actions become sticky bottom buttons or sheets.
- **Dark mode**: P3 (design tokens ready via CSS variables from day 1).
- **Accessibility**: visible focus rings, WCAG AA contrast, all interactive elements keyboard reachable, table row actions in a menu (not hover-only), form errors announced (`aria-describedby`), dialogs focus-trapped.

2. Global conventions (defined once — screens below note only deltas)

- **List screens**: toolbar (search input, filter dropdowns, sort select) → server-paginated DataTable (pageSize 20, column sort, total count) → row actions menu → bulk-select only where noted. **Empty state**: icon + one-line message + primary CTA. **Loading**: skeleton rows. **Error**: inline alert + Retry.
- **Forms**: zod-validated (same schema as API), inline field errors, submit disabled while pending, sticky footer bar (Cancel/Save) on long forms, unsaved-changes guard.
- **Detail screens**: header card (avatar/title/status badge/key facts + actions) → tabs.
- **Destructive actions**: confirm dialog naming the object; type-to-confirm for irreversible (delete employee, approve payroll).
- **Feedback**: success/error toasts; long operations show progress state on button.
- **Dates/times** render in company timezone with user-familiar format from settings.

3. Public screens

Screen	Spec
`/login`	Centered card: logo, email, password, "Forgot password?" link, submit. Errors: invalid credentials (generic message), account locked (with retry-after), account disabled. No signup link. Mobile: full-width card.
`/forgot-password`	Email field → always shows "If the account exists, we sent a link" (no enumeration).
`/reset-password`	Token from URL (also serves invite-accept with kind param): new password + confirm, live policy checklist (length, breach check note), expired/used-token error state with "request new link" CTA.

4. Employee screens

`/dashboard`

Purpose: daily landing. Widgets: **Check-in card** (big In/Out button, live worked-hours timer, today's shift, late badge), **Leave balances** (per-type chips), **Upcoming holidays** (next 3), **My pending items** (corrections pending, onboarding/offboarding tasks P2, reviews due P2), **Announcements** (latest 3). Quick actions: Apply Leave, Raise Correction. Mobile: single column; check-in card first.

`/me/profile`

Header card (photo, name, code, designation, department, manager, status badge). Tabs: **Personal** (self-editable: phone, personal email, address, photo — edit inline; rest read-only), **Job** (read-only placement/dates), **Emergency contacts** (CRUD own), **Bank** (P2; masked, read-only; "contact HR to change"). Resignation button (P2) opens offboarding dialog.

`/me/attendance`

Today card (punches list, worked total) + **month calendar grid** (color-coded day cells by status; legend; click day → drawer: punches, computed record, "Request correction" CTA). Month picker; summary bar (present/absent/half/leave/OT totals). Corrections list tab (status chips; cancel pending). Mobile: calendar becomes weekly horizontal scroll.

`/me/leave`

Balance cards per type (current + used/accrued). Requests table (dates, type, days, status, approver, actions: cancel). **Apply dialog**: type select (shows balance), date range, half-day toggle (single-day only), computed working days preview (live, holiday-aware), reason, attachment (if type requires). Errors surfaced inline: insufficient balance, overlap, notice period.

`/me/documents` **(P2)**

Grid/list of own + company docs (name, category, expiry badge, uploaded date). Upload button (allowed categories only), download, preview for images/PDF. Filter by category. Empty state: "No documents yet."

`/me/payslips` **(P2)**

Table: period, gross, net (amounts visible only here + HR), published date, Download PDF. Year filter. Empty: "Payslips appear after payroll is published."

`/me/expenses` **(P2)**

Table (date, category, amount, status chips) + New expense (form: category with limit hint, amount, date, description, receipt upload). Draft rows editable/submittable; rejected show reason; totals summary cards (pending/approved/reimbursed this year).

`/me/training` **(P3)**

Enrolled courses cards (progress ring, due badge) + course catalog (enroll). Course player page: lesson list sidebar, content pane, mark-complete, certificate download at 100%.

`/me/performance` **(P2)**

Goals list (progress sliders, status) + add goal (pending manager approval badge). Reviews: current cycle card → self-review form (rating + comments), history list.

`/me/notifications`

Full list (unread bold, type icon, timestamp, deep link), filters (unread/all, type), Mark all read. Pagination.

`/me/settings`

Change password (current + new + confirm, policy checklist). Session info + "Log out everywhere". Notification preferences (P3).

---

5. Manager screens (scope: direct reports; visible only with `*.view_team`/approve permissions)

`/team`

KPI row: team size, present today, on leave today, late today, pending approvals count. **Approvals inbox** (unified list: leave, corrections; P2 expenses, resignations — inline Approve/Reject with note dialog). Team list (person, status today, designation). Upcoming team leave (7-day mini calendar). Birthdays/anniversaries (day+month only).

`/team/attendance`

Date picker (default today) → grid: rows = reports, columns = status/in/out/worked/late; month view per employee (drill into same calendar component as `/me/attendance`, read-only + approve corrections). Filters: status. Export (P2, `reports.view_team`).

`/team/leave-approvals`

Pending queue (oldest first, SLA badge >48h) with request detail drawer (balance context: current balance, team members off same dates). Approve/Reject (note required on reject). History tab.

`/team/performance` (P2)

Reports' goals awaiting approval; manager reviews to complete (per cycle progress bar); per-report goal/review drill-down.

`/team/reports` (P2)

Team-scoped versions of R1–R7 report pages (see `03-modules-platform-and-reports.md` catalog) with export.

---

6. HR screens (permission `*.view_all` family)

`/hr` (HR Dashboard)

Composition per `03-modules-platform-and-reports.md` Module 18: KPI cards row, charts row (headcount by dept bar, 30-day attendance line, leave-type donut), action lists (absent-no-leave today, pending approvals oldest-first, probation ends, recent joiners, upcoming holidays), quick actions. All cards deep-link to filtered module screens.

`/hr/employees`

DataTable: photo+name, code, department, designation, location, type, status badge, join date. Filters: department, location, status, type; search name/code/email; sort any column; bulk export. Primary CTA: Add Employee (multi-step form: Personal → Job → Invite user toggle). Row actions: view, edit, deactivate. Pipeline filter chips: onboarding / on_notice (P2 views).

`/hr/employees/[id]`

Header card + status-transition action menu. Tabs: **Overview** (personal+contacts), **Job** (placement, manager, shift assignment with history, dates), **Attendance** (embedded month view + manual entry + corrections), **Leave** (balances + adjust dialog + history), **Salary** (P2 — assignment history, revise action; `payroll.*` only), **Documents** (P2), **Onboarding/Offboarding** (P2 checklists), **History** (audit trail for this employee).

`/hr/departments`

Two tabs: Departments (name, code, head, headcount, actions) and Designations (title, code, level). Inline create/edit dialogs. Delete guarded (409 explanation if in use).

`/hr/attendance`

Company day view (all employees, same grid as team but full scope + manual entry) | Corrections queue (all) | **Month lock panel**: month list with lock status, Lock button (type-to-confirm; validates no pending corrections — warning list if any) | Shift settings (shift CRUD + default marker).

`/hr/leave`

Tabs: Requests (all, filters: status/type/department/date), Balances (per-employee grid, adjust with reason), Types & Policies (CRUD forms with policy fields incl. sandwich toggle), Holidays (year view CRUD, location filter).

`/hr/payroll` (P2)

Runs list (period, status chip, gross/net totals). **Run wizard**: 1) Create (pick month; blocked if attendance month unlocked — checklist panel) → 2) Processing (progress) → 3) Review (lines table: employee, payable days, LOP, gross, deductions, net; expand → items; edit adjustments; recalculate) → 4) Approval (summary + type-to-confirm approve) → 5) Published (payslip statuses, notify-all, mark-paid). Settings tabs: Components, Structures. Employee salary assignment lives on employee detail Salary tab.

`/hr/documents` (P2)

All documents table (owner, name, category, expiry, status), Expiring-soon saved view (≤30d), category admin, upload-for-employee.

`/hr/recruitment` (P2)

Jobs list → job detail with **kanban** (columns = stages, cards = candidates: name, rating avg, days-in-stage; drag between stages with guard dialogs) | Candidates table (talent pool, search) | Interviews (calendar + my-interviews list for interviewer feedback form: rating, recommendation, notes) | Offers (status list; create-offer form; record accept → "Convert to employee" CTA pre-fills Add Employee).

`/hr/performance` (P2)

Cycles admin (create/activate/close), completion matrix (rows employees, cols self/manager/final), distribution chart, export.

`/hr/training` (P3)

Courses admin (CRUD + lessons editor with drag order), enrollment matrix (assign by dept/individual), completion report.

`/hr/reports`

Catalog grid (report cards by group) → report page pattern: filter bar (date preset, department, location, employee, status) + KPI strip + DataTable + Export CSV/XLSX (async >5k rows → toast "You'll be notified").

---

7. Admin screens

Screen	Spec
`/admin/company`	Company profile form (name, legal, logo upload, address, contact) + org defaults (timezone, currency, date format, working days, leave-year start).
`/admin/roles`	Roles list (system badge) → role detail: permission matrix grouped by module with toggle grid (system roles read-only in P1/P2; custom roles P3). Users tab: assign/remove roles per user, invite status, resend invite, disable user.
`/admin/locations`	Locations CRUD table.
`/admin/settings`	Grouped tabs per settings catalog (`03-…` Module 23): General, Attendance, Leave, Security (global keys visible only to super_admin), Notifications, Payroll (P2). Each change confirmable + audited note.
`/admin/audit-logs`	Filter bar (date range, actor, action, entity type, entity id) + table (time, actor, action, entity, summary) → detail drawer with before/after diff viewer (JSON, changed keys highlighted). Export.

---

8. Notification UX

- **Bell** (topbar): unread badge (9+ cap), dropdown of latest 10 (title, relative time, unread dot), item click = mark read + deep link; footer "View all" → `/me/notifications`; "Mark all read".
- In-app arrival: badge updates on poll (30s) — no websockets in v1.
- Announcements render as dismissible banner on `/dashboard` until read (receipt recorded).

---

9. Key composite screens — extra detail

Attendance month grid (shared component): 7-col calendar; cell = date, status color strip, worked hours; badges: late (amber dot), OT (blue dot), needs_review (red outline), locked (padlock watermark month-level). Hover/tap → punch summary popover.

Leave approval drawer: request facts, computed days breakdown (which days counted/skipped and why — sandwich/holiday notes), requester's balance after approval, team-absence conflict list, approve/reject buttons with note field.

Payroll review line expand: earnings/deductions item table + input snapshot (payable days math trace: period − LOP, proration factor) so HR can verify any number.

Employee add form (multi-step): validates per-step; step 3 "Invite user account now?" toggle; success screen with next actions (assign shift, start onboarding P2).

Decisions made in this document

- No websockets in v1 — 30s polling for the bell keeps infra simple; revisit with push in P3.
- Employee mobile bottom-nav items: Dashboard, Attendance, Leave, Notifications, Profile.
- Global search (topbar) is P2, scoped to employees (HR) — not a v1 blocker.
- Payslip amounts and any salary figures never appear in list previews or notifications — only inside `/me/payslips`, employee Salary tab (permission-gated), and payroll screens.
- All charts use Recharts with a shared theme wrapper; empty-data charts render a friendly placeholder, never an empty axis box.

HRMS Blueprint — Workflows, Notifications & Automation

> Document 07 of 11. States use canonical enums from `04-database.md`. Every transition below implies: permission check → transaction → audit log entry → notification(s) → (maybe) job enqueue.

1. Workflow maps

1.1 Employee onboarding (Candidate → Active Employee) — P2 (P1 = direct employee creation, steps 4–7)

```
1. Recruitment: offers.status = accepted
2. HR clicks "Convert to employee"
   → employees row created: status=onboarding, candidate_id set, join_date=offer.joining_date
3. Resume/offer docs copied as documents rows (category: contract/resume)
4. HR picks onboarding template → onboarding_tasks instantiated
   (assignees resolved: hr→HR user, manager→employee.manager.user, new_hire→employee.user, it→designated user;
    due_date = join_date + due_offset_days)
5. HR triggers user invite (users row status=invited, invite email)
6. Assignees complete tasks (status pending→completed; skip requires reason)
7. All required tasks completed → HR "Activate"
   → employees.status = active; welcome notification
```

- Actors: hr_admin (drive), manager/IT/new hire (tasks).
- Side effects: notifications on task assignment + overdue; audit on conversion/activation.
- Edge cases: offer declined after conversion started → HR cancels: employee row soft-deleted if never active, tasks cancelled. Join date shifts → due dates recompute (HR edits join_date; task recompute job).

1.2 Attendance day cycle — P1

```
check-in (punch in, source=web) ─▶ [0..n punch pairs] ─▶ check-out (punch out)
        nightly attendance-daily-calc (00:30) for yesterday:
        for each active employee:
          shift = effective employee_shifts row
          if approved leave covering date  → status=on_leave (half-day leave + punches → half_day/present math)
          elif holiday (location-aware)    → status=holiday
          elif weekday in shift.week_off   → status=week_off
          elif no punches                  → status=absent (notify)
          else: worked=Σ(in→out)−break; late=max(0, first_in−(start+grace));
                status = worked < half_day_threshold ? half_day : present;
                overtime = max(0, worked − shift_duration); missing check-out → needs_review
```

Correction sub-flow:

```
employee submits attendance_corrections (pending)
  ├─ employee cancel → cancelled
  ├─ manager reject  → rejected (note required, notify)
  └─ manager approve → approved → punches adjusted/created (source=manual, audit)
                       → day recomputed immediately
Guards: work_date in locked month → 422 at submit AND at approve; approver must be
        target's manager or hold attendance.approve with view_all (HR).
```

Month lock: HR locks (company, year, month) → all records flagged locked; punches/corrections/manual entries for that month rejected; unlock = super_admin-grade HR action, audited, only while payroll run for the month is absent or draft.

1.3 Leave — P1

State machine (`leave_requests.status`):

```
            ┌──────────── employee cancel ────────────┐
submit ─▶ pending ── manager/HR approve ─▶ approved ── cancel(before start, or HR) ─▶ cancelled(+restore)
            └── manager/HR reject ─▶ rejected
```

- Submit guards: dates valid & future-or-today (HR on-behalf may backdate), computed days > 0, balance ≥ days (unless policy max_negative), no overlap with pending/approved, min notice respected, probation rule, attachment if type requires.
- Approve (permission `leave.approve`, approver ≠ requester, target is direct report unless HR): re-check balance & overlap → decrement `leave_balances.used` (+days) atomically → notify employee → attendance for those dates will compute `on_leave`.
- Reject: note required → notify.
- Cancel pending: by employee anytime. Cancel approved: employee before start_date; HR anytime (past cancellations trigger attendance recompute for affected days) → restore balance.
- Edge cases: request spanning leave-year boundary → split across two balance years at submit; holiday/week-off inside span excluded unless sandwich rule on; balance changed between submit and approve → approval fails 422 with explanation.

1.4 Payroll — P2

```
create run (year, month) ── guards: attendance_month_locks row exists; no existing run
  draft ──"Process"──▶ processing (job): snapshot per employee:
        period_days, payable_days = period_days − LOP_days,
        LOP = absent days + unpaid-leave days (from locked attendance + approved unpaid leave),
        OT approved minutes, adjustments (manual + approved expense reimbursements opted-in)
        → payslips(draft) + payslip_items computed via payroll engine
  processing ──auto──▶ pending_approval
  pending_approval ──"Revert"──▶ draft (payslips deleted, inputs editable)
  pending_approval ──"Approve" (payroll.approve, type-to-confirm)──▶ approved
        → payslip-generate job: PDFs rendered, payslips.status=published, employees notified
  approved ──"Mark paid" (+paid_at)──▶ paid → payslips.status=paid
```

- Guards: employee joined mid-month → prorated by join-date; exited employees included only up to exit_date (final-settlement runs handle the rest); salary assignment missing → line flagged, run cannot leave pending_approval until resolved or employee excluded (explicit, audited).
- Rollback rules: anything before `approved` is reversible; `approved` is not — corrections happen via next-cycle adjustments (audited).

1.5 Offboarding — P2

```
employee submits resignation ─▶ offboarding_requests: initiated
  manager approves ─▶ HR confirms: sets notice_period_days (default from settings/employee),
                      last_working_day computed/overridable
                      → employees.status = on_notice ─▶ request: in_progress
                      → offboarding template instantiated (offboarding_tasks; blocking flags)
  tasks completed (asset return, access revocation, KT, exit interview)
     all blocking tasks done ─▶ HR marks cleared
  HR records settlement (leave encashment days×rate, pending reimbursements,
     recoveries, notice shortfall) → payroll settlement line ─▶ settled
  on/after last_working_day: HR completes ─▶ completed:
     users.status=disabled, session_version++, employees.status=exited, exit_date set
```

- Withdrawal: while initiated/in_progress and before last_working_day, employee requests, HR approves → request closed (kept for history), employee back to active, tasks cancelled.
- Edge cases: pending approved future leave beyond last working day → auto-cancelled with balance restore at confirmation; pending expenses → resolved before settled; employee is a manager → HR must reassign reports before completed (guard

422).

1.6 Recruitment — P2

```
job_postings: draft → open (publish) → on_hold ⇄ open → closed
application stages: applied → screening → interview → offer → hired
                      └──────────────── rejected (any stage, reason required)
interview: scheduled → completed (feedback+rating) | cancelled
offer: draft → sent (requires recruitment.approve) → accepted → [convert to employee]
                                  ├─ declined
                                  └─ withdrawn
```

- Guards: stage `offer` requires ≥1 completed interview (configurable); `hired` set automatically when offer accepted; converting requires stage=hired; closing a job with open applications prompts bulk-reject with reason.
- Side effects: interviewer notified on schedule; HR notified on feedback submitted; candidate emails (offer sent) are manual in P2 (mailto/attachment), automated P3.

1.7 Training — P3

```
course: draft → published → archived
enrollment: enrolled → in_progress (first lesson completed) → completed (all lessons; score if assessment)
    completed → certificate generated (PDF job) → notification
```

- Mandatory course assignment sets due_date = assigned_at + deadline_days → reminder cron.
- Edge cases: course content changed after enrollments → progress preserved by lesson id; archived course blocks new enrollment, preserves history.

---

2. Cross-cutting transition rules

- Every approval action: permission + scope check, approver ≠ subject, state precondition, atomic transaction, audit entry with before/after status, notification to affected parties.
- Invalid transitions return 422 BUSINESS_RULE with a machine-readable `details.transition` field.
- All state changes emit domain events (`module.event`) consumed by the notification dispatcher and audit writer (see `05-architecture.md` §3, §6).

---

3. Notifications & automation catalog

Channels: **A** in-app, **E** email, **P** push (P3). Trigger: **ev** domain event, **cr** cron.

Event key	Trigger	Recipients	Channels	Timing	Phase
`employee.invited`	ev	new user	E	instant	P1
`leave.submitted`	ev	approver (manager), HR toggle	A,E	instant	P1
`leave.approved` / `leave.rejected`	ev	requester	A,E	instant	P1
`leave.cancelled`	ev	approver	A	instant	P1
`attendance.correction_submitted`	ev	approver	A,E	instant	P1
`attendance.correction_resolved`	ev	requester	A,E	instant	P1
`attendance.late`	cr (daily-calc)	employee; manager per toggle	A	00:30 next day	P1
`attendance.absent_no_leave`	cr (daily-calc)	employee, manager	A,E	00:30 next day	P1

Event key	Trigger	Recipients	Channels	Timing	Phase
`attendance.missing_checkout`	cr (daily-calc)	employee	A	00:30 next day	P1
`attendance.month_locked`	ev	all HR users	A	instant	P1
`holiday.upcoming`	cr	all employees (location-aware)	A	T-1 08:00	P1
`birthday / work_anniversary`	cr	team members + HR (toggles)	A	08:00	P1
`probation.ending`	cr	HR, manager	A,E	T-7	P1
`payslip.published`	ev	employee	A,E	instant	P2
`payroll.pending_approval`	ev	payroll approvers	A,E	instant	P2
`document.expiring`	cr	owner employee, HR	A,E	T-30, T-7, T-0	P2
`contract.expiring`	cr	HR	A,E	T-30, T-7	P2
`expense.submitted`	ev	approver	A,E	instant	P2
`expense.approved/rejected/reimbursed`	ev	claimant	A,E	instant	P2
`onboarding.task_assigned / offboarding.task_assigned`	ev	assignee	A,E	instant	P2
`boarding.task_overdue`	cr	assignee, HR	A	daily 08:00	P2
`resignation.submitted`	ev	manager, HR	A,E	instant	P2
`offboarding.state_changed`	ev	employee, HR	A	instant	P2
`review.deadline`	cr	pending reviewers	A,E	T-7, T-1	P2
`goal.approved`	ev	employee	A	instant	P2
`interview.scheduled`	ev	interviewer	A,E	instant + T-1 reminder cr	P2
`announcement.published`	ev	audience users	A (+E toggle)	instant	P1
`training.deadline`	cr	enrollee, manager	A,E	T-7, T-1	P3
`training.completed`	ev	enrollee (certificate)	A,E	instant	P3
`report.export_ready`	ev	requester	A	instant	P2

Automation policy: instant events dispatch immediately (email via queue); cron-driven events batch into one job run each (see job table in `05-architecture.md` §7). Digest mode (P2): user-opted events collapse into 18:00 daily email instead of instant emails; in-app always instant. Quiet hours: emails only 07:00–21:00 company TZ — outside window, deferred to window start (in-app unaffected).

Decisions made in this document

- Leave requests spanning leave-year boundary are split into two linked requests at submit (simpler balance math, clearer approvals).
- Correction approval edits/creates punches (source `manual`) rather than editing the computed record — the record stays derivable from punches at all times.
- Payroll `approved` state is immutable by design; monetary corrections are next-cycle adjustments — matches audit expectations.
- Offer-stage automation (candidate-facing emails) deferred to P3; P2 records statuses while HR communicates manually.
- Cron times are company-timezone-relative; multi-tenant P3 runs per-company schedules from one scheduler loop.

HRMS Blueprint — API Specification

> Document 08 of 11. Canonical endpoint catalog. Shapes reference `04-database.md` ; workflow guards reference `07-workflows-and-automation.md` .

1. Conventions

- Base `/api/v1` ; plural kebab-case resources. Session-cookie auth (JWT, httpOnly). CSRF: SameSite=Lax cookie **plus** Origin/Referer check on all mutating methods (reject mismatches 403).
- Pipeline per route: authenticate → tenant resolve → `requirePermission` → zod validate → service (see `05-architecture.md` §3).
- Success `{ data, meta? }` ; list meta `{ page, pageSize, total }` . Pagination `?page=1&pageSize=20` (max 100). Filtering `?q=&sort=field:asc&from=&to=&departmentId=…` .
- Error `{ error: { code, message, details? } }` .

Common errors (defined once; apply everywhere)

HTTP	code	When
400	VALIDATION_ERROR	zod failure; `details` = field map
401	UNAUTHENTICATED	no/expired session, session_version mismatch
403	FORBIDDEN	missing permission, scope violation, origin mismatch
404	NOT_FOUND	id absent **or outside tenant scope** (no existence leak)
409	CONFLICT	unique violation, delete-guard (referenced rows)
422	BUSINESS_RULE	workflow guard (named in `details.rule`)
429	RATE_LIMITED	bucket exceeded; `Retry-After` header
500	INTERNAL	unexpected; generic message only

Per-endpoint tables below list only **permission, request essentials, response essentials, and notable 422/409 rules**. CRUD validation (types/lengths/enums) follows `04-database.md` implicitly.

2. Auth (Phase 1) — public unless noted

Method & path	Perm	Request	Response	Notable errors
POST `/auth/login`	—	email, password	user summary + permissions; sets cookie	401 generic invalid-credentials; 429; 422 `account_locked` , `account_disabled`
POST `/auth/logout`	session	—	ok; clears cookie	
POST `/auth/forgot-password`	—	email	always ok (no enumeration)	429
POST `/auth/reset-password`	—	token, password	ok	422 `token_invalid_or_expired` , `password_policy`
POST `/auth/accept-invite`	—	token, password	ok (activates user)	same as reset
GET `/auth/me`	session	—	user, employee summary, roles, permissions, company	

3. Users, roles, permissions (Phase 1)

Method & path	Perm	Notes
GET `/users`	`users.view_all`	filters: status, role
POST `/users/:id/resend-invite`	`users.manage`	422 `not_invited_state`
POST `/users/:id/disable` / `enable`	`users.manage`	disable bumps session_version; 422 `cannot_disable_self`
GET `/roles`	`roles.view_all`	with permission codes
POST `/roles` / PATCH `/roles/:id`	`roles.manage`	P3 (custom roles); system roles immutable 422 `system_role`
PUT `/roles/:id/permissions`	`roles.manage`	replace grant set; audited
GET `/permissions`	`roles.view_all`	global catalog
POST `/users/:id/roles` / DELETE `.../roles/:roleId`	`roles.manage`	422 `last_hr_admin` guard

4. Company, settings, locations (Phase 1)

Method & path	Perm
GET/PATCH `/companies/current`	view: session; PATCH: `company.manage`
GET `/settings` · PUT `/settings/:key`	`settings.manage` (global keys: super_admin only → 403)
GET/POST `/locations` · PATCH/DELETE `/locations/:id`	view: session; write: `company.manage` ; DELETE 409 if employees assigned

5. Departments & designations (Phase 1)

Method & path	Perm
GET/POST `/departments` · PATCH/DELETE `/departments/:id`	view: session; write: `department.manage` ; DELETE 409 in-use
GET/POST `/designations` · PATCH/DELETE `/designations/:id`	same pattern

6. Employees (Phase 1; bank P2)

Method & path	Perm	Notes
GET `/employees`	`employee.view_all` | `view_team` (auto-scoped)	filters: departmentId, locationId, status, type, q
POST `/employees`	`employee.create`	multi-section body; optional `invite:true` ; 409 duplicate code/email
GET `/employees/:id`	`view_all` | `view_team` (report) | `view_own` (self)	DTO strips salary/bank unless self/payroll perms
PATCH `/employees/:id`	`employee.edit` ; self-service subset via `employee.view_own` (allowlisted fields)	403 on non-allowlisted self edit
POST `/employees/:id/status`	`employee.edit`	{status}; 422 invalid transition (e.g. exited→active)
DELETE `/employees/:id`	`employee.delete`	soft; 422 `has_active_user` (disable first)

Method & path	Perm	Notes
GET/POST/PATCH/DELETE `/employees/:id/emergency-contacts(/:cid)`	own or `employee.edit`	
GET/PUT `/employees/:id/bank-account`	self read (masked) or `payroll.manage`	P2; write audited
POST `/employees/:id/invite`	`users.manage`	creates user+invite; 409 user exists
GET `/employees/export`	`employee.export`	CSV; audited

7. Shifts (Phase 1)

Method & path	Perm	Notes
GET/POST `/shifts` · PATCH/DELETE `/shifts/:id`	view: session; write `shifts.manage`	DELETE 409 if assigned; one default enforced 422
GET `/employee-shifts?employeeId=`	scope-tiered	assignment history
POST `/employee-shifts`	`shifts.manage`	{employeeId, shiftId, effectiveFrom}; auto-closes previous; 422 `overlap`

8. Attendance (Phase 1)

Method & path	Perm	Request → response	Notable errors
POST `/attendance/check-in` · `/check-out`	`attendance.view_own` (self only)	→ punch + today summary	422 `already_checked_in` / `not_checked_in` (idempotency), `month_locked`, `not_active_employee`
GET `/attendance/records`	own/team/all tier params	employeeId?, month, → records+punches	403 scope
POST `/attendance/records` (manual)	`attendance.edit`	employeeId, date, in/out	422 `month_locked`; audited
GET `/attendance/summary`	tier	month aggregates (present/absent/...)	
GET/POST `/attendance/corrections`	own create; list per tier	date, requested in/out/status, reason	422 `month_locked`, `duplicate_pending`, `future_date`
POST `/attendance/corrections/:id/approve` · `/reject`	`attendance.approve` (team or all)	reject: note required	422 `not_pending`, `own_request`, `month_locked`
POST `/attendance/corrections/:id/cancel`	requester		422 `not_pending`
GET/POST `/attendance/locks`	`attendance.manage`	{year, month}	422 `pending_corrections_exist` (list in details), 409 already locked
DELETE `/attendance/locks/:id`	`attendance.manage`	unlock	422 `payroll_run_exists`

9. Leave & holidays (Phase 1)

Method & path	Perm	Notable errors
GET/POST `/leave-types` · PATCH/DELETE `/leave-types/:id`	write `leave.manage`	DELETE 409 in-use
GET/PUT `/leave-policies/:leaveTypeId`	`leave.manage`	
GET `/leave-balances`	tier (own/team/all; employeeId, year)	
POST `/leave-balances/adjust`	`leave.manage`	{employeeId, typeId, year, delta, reason}; audited
GET `/leave-requests`	tier + filters status/type/date	
POST `/leave-requests`	own `leave.create` (HR on-behalf: `leave.create` + `view_all`, employeeId param)	422 `insufficient_balance`, `overlap`, `min_notice`, `probation`, `attachment_required`, `max_consecutive`, `invalid_range`
POST `/leave-requests/:id/approve` · `/reject`	`leave.approve` (scope)	422 `not_pending`, `own_request`, `balance_changed`; reject note required
POST `/leave-requests/:id/cancel`	requester (rules) or HR	422 `already_started` (non-HR)
GET `/leave-requests/:id/day-breakdown`	request visibility	computed day list with skip reasons
GET/POST `/holidays` · PATCH/DELETE `/holidays/:id`	write `holidays.manage`	409 duplicate date+location

10. Payroll (Phase 2)

Method & path	Perm	Notable errors
CRUD `/payroll/components`, `/payroll/structures` (+ `/components` sub-list)	`payroll.manage`	409 code dup; 422 `percentage_base_missing`
GET/POST `/payroll/employee-salaries`	`payroll.manage` (GET own summary: none — employee never sees structure admin, only payslips)	POST: {employeeId, structureId?, components, ctc, effectiveFrom}; 422 `overlap`
GET/POST `/payroll/runs`	`payroll.view_all` / `payroll.create`	POST 422 `month_not_locked`, 409 run exists
POST `/payroll/runs/:id/process`	`payroll.create`	422 `not_draft`, `missing_salary_assignments` (details list)
POST `/payroll/runs/:id/revert`	`payroll.create`	422 `not_pending_approval`
PATCH `/payroll/runs/:id/lines/:payslipId`	`payroll.edit`	draft-run adjustments only 422
POST `/payroll/runs/:id/approve`	`payroll.approve`	422 `not_pending_approval`, `unresolved_lines`; irreversible
POST `/payroll/runs/:id/mark-paid`	`payroll.approve`	422 `not_approved`
GET `/payroll/payslips`	`payroll.view_all`; `?employeeId=me` via `payroll.view_own`	
GET `/payroll/payslips/:id/pdf`	owner or `payroll.view_all`	presigned redirect; audited

11. Documents (Phase 2)

Method & path	Perm	Notable errors
CRUD `/document-categories`	`documents.manage`	
GET `/documents`	tier (own + company-level; manager: category.manager_visible only)	filters: categoryId, employeeId, expiring≤days
POST `/documents/presign-upload`	own (uploadable categories) or `documents.create`	{categoryId, employeeId?, filename, mime, size} → presigned PUT + draft key; 422 `category_not_uploadable`, `mime_not_allowed`, `too_large`
POST `/documents` (confirm)	same	metadata row; 422 `upload_not_found`
GET `/documents/:id/download`	object-level check	presigned GET redirect; sensitive categories audited
PATCH/DELETE `/documents/:id`	`documents.edit` / `delete`	system_managed 422

12. Recruitment (Phase 2)

Method & path	Perm	Notable errors
CRUD `/job-postings`; POST `/:id/publish` · `/close`	`recruitment.*`	422 close-with-open-apps prompts `open_applications`
CRUD `/candidates`	`recruitment.*`	409 email dup
GET/POST `/applications`	`recruitment.view_all/create`	409 candidate+job dup
POST `/applications/:id/stage`	`recruitment.edit`	{stage, reason?}; 422 `invalid_transition`, `interview_required`, reason required for rejected
CRUD `/interviews`; POST `/interviews/:id/feedback`	schedule `recruitment.edit`; feedback: assigned interviewer	422 `not_interviewer`, `already_submitted`
POST `/offers` · `/offers/:id/send` · `/accept` · `/decline` · `/withdraw`	create `recruitment.create`; send requires `recruitment.approve`	422 state guards, `expired`
POST `/offers/:id/convert-to-employee`	`employee.create`	422 `not_accepted`; returns new employeeId

13. Onboarding & offboarding (Phase 2)

Method & path	Perm	Notable errors
CRUD `/onboarding/templates` (+tasks)	`onboarding.manage` (kind on/offboarding)	
POST `/onboarding/tasks/instantiate`	`onboarding.manage`	{employeeId, templateId}
GET `/onboarding/tasks`	own-assigned or `onboarding.view_all`	
POST `/onboarding/tasks/:id/complete` · `/skip`	assignee or HR	skip reason required; 422 `not_pending`
POST `/employees/:id/activate`	`onboarding.manage`	422 `required_tasks_incomplete` (list)
POST `/offboarding/requests`	own (`offboarding.create`) or HR on-behalf	422 `already_active_request`
POST `/offboarding/requests/:id/approve`	manager `offboarding.approve`	

Method & path	Perm	Notable errors
POST `/offboarding/requests/:id/confirm`	`offboarding.manage`	sets last_working_day; instantiates tasks; auto-cancels post-LWD leave
POST `/offboarding/requests/:id/withdraw`	employee + HR approval flow	422 after LWD
POST `/offboarding/tasks/:id/complete`	assignee/HR	
POST `/offboarding/requests/:id/clear` · `/settle` · `/complete`	`offboarding.manage`	422 `blocking_tasks_pending`, `settlement_missing`, `manager_has_reports`

14. Performance (Phase 2)

Method & path	Perm	Notable errors
CRUD `/performance/cycles`; POST `/:id/activate` · `/close`	`performance.manage`	422 state guards
GET/POST/PATCH `/performance/goals`	own create/update-progress; approve `performance.approve` (manager)	422 `cycle_closed`
GET `/performance/reviews`	tier	
POST `/performance/reviews/:id/submit-self`	subject employee	422 `not_pending_self`
POST `/performance/reviews/:id/submit-manager`	manager of subject	422 `not_pending_manager`, `self_review_missing`
POST `/performance/reviews/:id/reopen`	`performance.manage`	422 `cycle_closed`

15. Training (Phase 3)

CRUD `/training/courses` (+ `/lessons`) — `training.manage`; publish guard `no_lessons`. GET `/training/courses` (published) — session. POST `/training/enrollments` — self-enroll open courses or `training.manage` assign; 409 dup. POST `/training/enrollments/:id/lessons/:lessonId/complete` — enrollee; auto-completes course at 100% → certificate job. GET `/training/certificates/:id/pdf` — owner/HR.

16. Expenses (Phase 2)

Method & path	Perm	Notable errors
CRUD `/expense-categories`	`expenses.manage`	
GET/POST/PATCH `/expenses`	own (draft edit only); tier list	422 `receipt_required`, `over_limit_needs_hr` flag
POST `/expenses/:id/submit`	owner	422 `not_draft`
POST `/expenses/:id/approve` · `/reject`	`expenses.approve` scope	422 `not_submitted`, `own_claim`, `over_limit_needs_hr`
POST `/reimbursements`	`expenses.manage`	{expenseIds, method, reference} → marks reimbursed; 422 `not_approved`

17. Notifications & announcements (Phase 1)

GET `/notifications` (own; unread filter) · POST `/notifications/:id/read` · POST `/notifications/read-all`. GET `/announcements` (audience-scoped) · POST `/announcements` (`announcements.create`; body sanitized) · POST `/announcements/:id/publish` · POST `/announcements/:id/read` (receipt).

18. Reports (Phase 1 basic → P2 full)

GET `/reports/:name` — name ∈ catalog (R1–R16, `03-…` doc); perm `reports.view_all` or `reports.view_team` (auto-scoped); query = report filters; → `{ data: rows, meta }`. POST `/reports/:name/export` — additionally `*.export` for the module; small sync (CSV stream) / large async (202 + notification); every call audited.

19. Audit logs (Phase 1)

GET `/audit-logs` — `audit.view_all` (company scope); filters actor, action, entityType, entityId, from/to. GET `/audit-logs/:id` — detail with before/after. (No write endpoints exist.)

Decisions made in this document

- HR "on-behalf" actions reuse the standard endpoints with an `employeeId` parameter + elevated permission, rather than separate admin endpoints.
- Check-in/check-out endpoints are self-only by design; manual third-party entries go through `/attendance/records` (manual, audited).
- 404 (not 403) for cross-tenant/out-of-scope ids — no existence leaking.
- Report names are a closed server-side registry; no ad-hoc query endpoint (SQL injection surface stays zero).
- Payslip PDF and sensitive-document downloads always route through the API for permission check + audit, never direct bucket links.

HRMS Blueprint — Security, Privacy & Data Protection

> Document 09 of 11. HR data is among the most sensitive a company holds (identity, salary, health-adjacent leave data, bank details). Security is a first-class requirement, not a hardening phase.

1. Threat model summary

Assets: employee PII (IDs, DOB, addresses), salary & bank data, documents (contracts, ID proofs), leave/health hints, credentials, audit trails. **Adversaries & scenarios**:

- Cross-tenant leakage (future SaaS): tenant A reads tenant B → mitigated by dual-layer tenancy (§4).
- Privilege escalation: employee → manager/HR data (IDOR, mass assignment) → object-level checks, DTO allowlists, permission middleware.
- Credential attacks: stuffing/brute force → argon2id, lockout, rate limits.
- Injected content: XSS via names/announcements, SQL injection → output encoding, sanitization, parameterized queries.
- Insider misuse: HR exporting everything quietly → export auditing, least-privilege permissions, four-eyes payroll.
- Infrastructure: stolen backups, leaked buckets → encryption at rest, private buckets, presigned-only access.

2. Authentication

- **Hashing**: argon2id (memory 64 MB, iterations 3, parallelism 1 — tune to ~100ms server budget). No MD5/SHA/bcrypt-cost-shortcuts.
- **Password policy**: min length 10 (configurable `security.password_min_length`), no arbitrary complexity rules; reject top-breached passwords (embedded top-100k list; HIBP k-anonymity check P3). Policy enforced in shared zod schema.
- **Lockout**: 5 failed attempts (configurable) → 15-min lock, progressive doubling; lockout events audited + notify user by email. Login errors stay generic (no user/password distinction).
- **Tokens (invite/reset)**: 32-byte random, stored SHA-256 hashed, single-use (`used_at`), TTL invite 7d / reset 1h; all active tokens invalidated on password change.
- **Sessions**: JWT in httpOnly Secure SameSite=Lax cookie; claims `{userId, companyId, sessionVersion}`; sliding 12h expiry; `session_version` bump = global logout (password change, disable, offboarding completion). No tokens in localStorage, ever.

3. Authorization

- All checks **server-side** in the `withApi` pipeline; UI checks are cosmetic only.
- Permission-based (`module.action`) — never role-name conditionals (see `00-overview-and-roles.md` §6).
- **Scope tiers** enforced in services: own (`employee_id = ctx.employeeId`), team (`manager_id = ctx.employeeId`), company (tenant scope + `view_all`), platform (super_admin via `adminDb`, every use audited).
- **Object-level guards** (beyond scope): approver ≠ requester; manager actions verify target's `manager_id`; user cannot disable self; last-`hr_admin` protection.
- **Mass-assignment defense**: zod schemas whitelist fields per endpoint; self-service employee edits restricted to the allowlisted field set.
- **Response filtering**: DTO mappers strip salary/bank/ID fields unless caller is self or holds payroll/document permissions — enforced in one mapper per module, not per call-site.
- Permission/role changes audited with before/after grant sets.

4. Row-Level Security & tenancy

Model detailed in `04-database.md` §4: tenant-scoped Prisma extension (primary) + Postgres RLS with `SET LOCAL app.company_id` (defense-in-depth), app DB role without `BYPASSRLS`. **What RLS does not cover** — and the app layer must: own/team tier filtering,

field-level exposure, object-level workflow guards, rate limiting, business rules. RLS is the isolation floor, not the authorization system. CI runs tenant-isolation tests with two seeded companies asserting zero cross-reads through both the API and raw-SQL paths (`10-roadmap-testing-deployment.md` §3).

5. Secure file access

- Buckets private; zero public ACLs; bucket policy denies non-presigned access.
- Presigned PUT TTL 10 min (content-type + max-size conditions signed in); presigned GET TTL 5 min; every GET preceded by object-level permission check; sensitive-category downloads audited.
- Filename sanitized (stored name is display metadata; key is server-generated `{companyId}/{module}/{entityId}/{uuid}`) — path traversal impossible by construction.
- Mime allowlist per document category; magic-byte sniff on confirm for executables masquerading as PDFs; size caps (10 MB default).
- EXIF stripped from profile photos on ingest. Virus-scan job hook (ClamAV) P3.

6. Input handling

- zod validation on **every** endpoint (body, query, params) — shared schemas with the frontend.
- SQL injection: Prisma parameterization everywhere; the report module's raw SQL uses bound parameters only, and report names resolve against a closed registry (no dynamic SQL from input).
- XSS: React auto-escaping; `dangerouslySetInnerHTML` allowed **only** for announcement bodies, which are sanitized server-side at write time (allowlist tags via sanitize-html) — render path never re-trusts input.
- CSV export cells prefixed to neutralize formula injection (`=` , `+` , `-` , `@`).
- File upload validation per §5.

7. CSRF

SameSite=Lax cookie blocks cross-site POSTs from foreign origins; belt-and-braces Origin/Referer verification on all mutating methods (403 on mismatch). No CORS allowances for `/api/v1` beyond same origin. This combination is sufficient because auth lives only in the cookie and all mutations are non-GET.

8. Rate limiting

Redis token buckets: per-IP `/auth/*` 10/min; per-user mutations 60/min; exports/reports 10/min per user; presign 30/min. 429 with `Retry-After` . Lockout (§2) is separate and account-based.

9. Secrets & configuration

- All secrets via env vars (catalog in `10-roadmap-testing-deployment.md` §4); `.env` git-ignored; `.env.example` documents every key with dummy values.
- CI: secret scanning (gitleaks) + dependency audit (npm audit / osv-scanner) gating merges.
- DB users: `app_user` (RLS-bound, INSERT-only on audit_logs, no DDL), `migrate_user` (DDL, CI only), `worker_user` (as app). No superuser at runtime.
- JWT secret rotation supported via keyid + dual-accept window.

10. Audit logging

Catalog and shape in `03-modules-platform-and-reports.md` Module 22. Security-relevant guarantees: append-only (no UPDATE/DELETE grants), actor + IP + user-agent captured, before/after diffs for sensitive fields (bank/salary diffs store masked values), **exports and sensitive downloads are themselves logged**, system jobs log under a system actor. Viewer restricted to `audit.view_all` .

11. Data protection & privacy

- **Field exposure rules** (canonical): salary/payslips → self + `payroll.*` holders only; bank account numbers → masked for self, full only to `payroll.manage` (and audited); identity documents → self + `documents.view_all`; DOB year hidden in team widgets; managers see work data only.
- TLS everywhere (HSTS); Postgres disk encryption; **column-level encryption** (AES-256-GCM, app-managed key) for `employee_bank_accounts.account_number_enc`.
- PII minimization: collect only fields with an HR purpose; candidate data purged per retention (§13).
- Backups encrypted (age/KMS) at rest and in transit.
- Access-on-request: employee sees own data through ESS by design; data export for a person (GDPR-style) = HR export of the employee profile (P3 automation).

12. Backup strategy

- Nightly `pg_dump` (custom format) to object storage + WAL archiving for PITR where the host supports it.
- Retention: 30 daily, 12 weekly, 12 monthly; encrypted; separate credentials from app.
- **Restore drill**: monthly scripted restore to staging + smoke test — a backup that hasn't been restored doesn't exist.
- Object storage: S3 versioning on the documents bucket; lifecycle to archive class at 90 days for soft-deleted keys.

13. Data retention (defaults; configurable `retention.*` settings)

Entity	Retention	After expiry
payslips / payroll_runs	7 years after fiscal year	archive export, then purge
attendance (punches+records)	3 years	aggregate to monthly summaries, purge detail
leave_requests / balances	3 years after year close	purge detail
candidates / applications (not hired)	1 year after last activity	purge PII, keep anonymized funnel stats
documents (exited employees)	per category (contracts 7y, ID proofs 1y post-exit)	purge files + rows
audit_logs	2 years	purge (export option first)
notifications	6 months	purge
exited employee core record	retained (soft-deleted state) for reporting; PII fields nullable-scrub at 7y	scrub

Retention jobs run weekly (worker), log what they purge, and never touch legal-hold-flagged rows (`legal_hold bool` — add to employees/documents when first needed, P3).

14. Security testing hooks (feeds `10-roadmap-testing-deployment.md`)

Must-have automated suites: permission matrix (every endpoint × role), tenant isolation (two companies), object-level (approve-own-request forbidden, manager-of-other-team forbidden, IDOR probes with foreign ids → 404), auth (lockout, token reuse, expired token, session-version logout), file access (foreign-tenant document id, presign tampering), rate-limit smoke, CSV-injection, sanitization snapshots. CI additionally runs dependency + secret scans. Annual external pentest before multi-tenant GA (P3 gate).

Decisions made in this document

- Argon2id parameters pinned above; revisit only with measured server budgets.
- Breach-password checking ships with an embedded top-100k list in P1 (offline, no third-party call); HIBP API is P3.
- Column-level encryption limited to bank account numbers in P1/P2 — broader field encryption deferred until a compliance driver exists (cost/benefit).

- No CAPTCHA: lockout + rate limiting + invite-only accounts make it unnecessary; revisit if public careers portal (P3) adds public forms.
- Monthly restore drills are a hard operational requirement, tracked in the go-live checklist.

HRMS Blueprint — Roadmap, Testing, Deployment & Build Order

> Document 10 of 11. Phase scope is canonical (matches `00-overview-and-roles.md` §3).

1. MVP rationale — what NOT to build in v1

MVP = a company can run daily HR ops: people, attendance, shifts, leave, holidays, approvals, dashboards, notifications, audit. Everything below is deliberately excluded from v1:

Excluded	Why
Payroll	Highest correctness risk; needs a full locked month of trustworthy attendance data first
Recruitment	Valuable but independent; zero coupling to daily ops
Onboarding/Offboarding modules	Manual HR process suffices at pilot scale; needs stable employee lifecycle first
Performance	Cycle-based; no urgency in first months
LMS	Biggest scope item with least operational urgency
Expenses	Payout depends on payroll; standalone value limited
Custom roles	4 system roles cover pilot; permission architecture already future-proofs it
SSO / MFA	Invite-only + strong passwords acceptable at pilot scale
PWA/push	Responsive web covers mobile day-1 needs
Biometric/integrations	Seams designed (`05-architecture.md` §8); hardware later
Advanced analytics / report builder	Fixed report catalog answers the real questions
Document-expiry automation	Documents module itself is P2

2. Development roadmap

Milestone template fields: Objective · Features · DB · Backend · Frontend · Security · Testing · Dependencies · **Done when**.

Phase 1 (MVP)

M0 — Foundation *(everything depends on this)*

- Objective: running skeleton with auth, RBAC, tenancy, audit plumbing.
- Features: login/logout/reset/invite, app shell, settings read.
- DB: initial migration — org (companies, locations, system_settings), auth tables, employees (minimal), audit_logs; **RLS policies + app_user roles in same migration set**; seed (permissions catalog, 4 roles, pilot company, admin user, default shift placeholder).
- Backend: `withApi` pipeline, tenantDb extension, error taxonomy, domain-event emitter, audit writer, mailer + queue wiring, `/api/health`, auth endpoints.
- Frontend: (auth) screens, (app) shell with permission-driven sidebar, empty dashboard.
- Security: argon2id, lockout, rate limits, CSRF origin check, secret scanning in CI.
- Testing: unit (permissions), integration (auth flows, tenant smoke with 2 companies), CI green.
- Done when: invited user logs in, sees empty shell; foreign-tenant read test returns zero rows; audit rows written for auth events.

M1 — Org & employees

- Features: locations, departments, designations, employee CRUD + profile + emergency contacts + invite, employee self-profile edit, `/hr/employees` + detail, `/me/profile`.
- DB: complete employees + emergency_contacts (+ soft-delete filter in client).

- Security: DTO field allowlists, self-edit allowlist, object-level 404 tests.
- Done when: HR creates employee → invite → employee edits own phone; permission matrix tests pass for these endpoints.

M2 — Shifts & attendance

- Features: shift CRUD + assignment, check-in/out, punches, nightly calc job, corrections + approvals, month lock, manual entry, `/me/attendance`, `/team/attendance`, `/hr/attendance`.
- DB: shifts, employee_shifts, attendance_* tables, attendance_month_locks.
- Backend: `attendance/calc.ts` pure calculator + worker jobs (`attendance-daily-calc`).
- Testing: heavy unit table for calc (late/half/OT/overnight/week-off/missing-checkout), correction workflow integration, lock guards.
- Done when: a simulated month of punches produces correct records; correction round-trip works; locked month rejects everything.

M3 — Leave & holidays

- Features: types/policies, balances + accrual + rollover jobs, requests + approvals + cancellation, holidays CRUD, `/me/leave`, `/team/leave-approvals`, `/hr/leave`.
- Backend: `leave/day-count.ts`, `leave/accrual.ts` pure; balance transactions.
- Testing: day-count unit table (sandwich on/off, half-day, year-boundary split, proration), balance restore paths, approve-own blocked.
- Done when: full leave lifecycle with correct balances incl. cancellation restore; attendance shows `on_leave`.

M4 — Dashboards, notifications, audit viewer, settings

- Features: notification service + templates + bell + center, announcements, employee `/dashboard`, `/team`, `/hr` dashboards, `/admin/*` (company, locations, roles view, settings, audit-logs viewer), P1 cron set (accrual, daily-calc, birthdays, probation), basic reports R1–R4, R6–R7, R16.
- Done when: every P1 catalog event fires both channels; dashboards show live data; audit viewer filters work.

M5 — Hardening & pilot launch

- Features: empty/loading/error polish, mobile pass, seed-for-pilot tooling.
- Security: full permission-matrix suite, isolation suite, rate-limit smoke, backup + restore drill executed.
- Testing: Playwright e2e (login, check-in, leave request→approve, correction→approve, month lock), load test attendance month view + dashboards (500-employee dataset).
- Done when: go-live checklist (§5) fully checked; pilot company onboarded.

Phase 2 (each milestone independently shippable)

- **P2-A Payroll**: components/structures/salaries, runs + engine + approval + payslip PDFs, `/hr/payroll`, `/me/payslips`, bank accounts (encrypted). Depends: M2 locks. Done when: parallel-run month matches manual payroll for pilot.
- **P2-B Documents + Expenses**: storage flows, categories, expiry scans; expense lifecycle + reimbursements. Done when: doc expiry notifies T-30/7/0; expense claim→reimbursed round-trip.
- **P2-C Recruitment + Onboarding + Offboarding**: pipeline/kanban, interviews, offers, conversion; templates/tasks/activation; resignation→settlement→deactivation. Done when: candidate→active-employee and employee→exited e2e pass incl. settlement handoff.
- **P2-D Performance**: cycles, goals, reviews, completion matrix. Done when: full cycle with reminders completes.
- **P2-E Reports & exports**: full catalog R5, R8–R15, async exports, digests. Done when: every report matches SQL-verified fixtures; exports audited.

Phase 3 (outline)

LMS → advanced analytics → automation rules → integrations (biometric first) → PWA/push → multi-tenant self-serve + custom roles + SSO → compliance extras (legal hold, HIBP, external pentest gate).

---

3. Testing strategy

Layer	Tooling	Coverage
Unit	Vitest	pure calculators (attendance calc, leave day-count, accrual, payroll engine), permission evaluation, zod schemas
Integration	Vitest + Testcontainers (Postgres+Redis)	services + API routes against real DB with RLS active
API contract	integration suite asserting envelope/error codes per `08-api.md`	every endpoint happy + guard paths
Permission matrix	generated suite: **every endpoint × 4 roles**	expected 2xx/403 grid derived from role matrix in `00-…` §6.4
Tenant isolation	two seeded companies	every list/detail endpoint + raw SQL probes: zero cross-reads; foreign ids → 404
Workflow/state	integration	every transition in `07-…` incl. **invalid** transitions → 422
E2E	Playwright	login, check-in/out, leave apply→approve, correction, month lock, (P2) payroll wizard, offboarding
Responsive	Playwright viewports (375px, 768px, 1280px)	employee core flows on mobile
Performance	k6 or artillery	attendance month view, dashboards, payroll run @ 500 employees < 5 min, report exports
Security	suites from `09-security.md` §14 + gitleaks + osv in CI	

HR-specific test cases (mandatory list): mid-month joiner accrual proration; leave overlapping holiday/week-off (sandwich on & off); half-day leave + worked half-day same date; overnight shift crossing midnight and DST-less TZ sanity (company TZ fixed); month lock blocks punches/corrections/manual entries; negative-balance prevention & max_negative honored; payroll rerun blocked after approve, revert allowed before; LOP combining absence + unpaid leave; approve-own-request blocked (leave/correction/expense); manager cannot act on non-reports; exited employee: login blocked, sessions dead, excluded from payroll after exit date; duplicate check-in idempotent; leave request spanning leave-year boundary splits correctly; balance changed between submit and approve → 422; correction on future date rejected.

---

4. Deployment & operations

Environments: dev (docker-compose full stack incl. Mailpit/MinIO) → staging (prod-shape, masked seed data, every migration rehearses here) → prod.

Env var catalog (secret ⚿): `DATABASE_URL` ⚿, `DIRECT_DATABASE_URL` ⚿(migrations), `REDIS_URL` ⚿, `AUTH_SECRET` ⚿, `APP_URL`, `S3_ENDPOINT`, `S3_BUCKET`, `S3_ACCESS_KEY_ID` ⚿, `S3_SECRET_ACCESS_KEY` ⚿, `EMAIL_PROVIDER`, `RESEND_API_KEY` ⚿, `EMAIL_FROM`, `SENTRY_DSN`, `FIELD_ENCRYPTION_KEY` ⚿(bank numbers), `SEED_DEMO`, `LOG_LEVEL`, `NODE_ENV`.

Migrations: `prisma migrate deploy` as a CI step before app rollout; backward-compatible releases (expand-contract for destructive changes); staging rehearsal mandatory; RLS/constraint SQL versioned with migrations.

CI/CD (GitHub Actions): lint → typecheck → unit → integration (testcontainers) → build images → push → staging deploy + migrate + smoke (health, login, isolation probe) → manual gate → prod migrate + deploy → smoke → tag release.

Monitoring: uptime checks on `/api/health` (readiness = DB+Redis+queue depth); alerts: p95 latency, 5xx rate, job failure count, queue depth > threshold, cron-missed heartbeats (dead-man switch on daily-calc). Logs: pino JSON → host aggregation (Loki/CloudWatch). Errors: Sentry web+worker, release-tagged.

Backups: per `09-security.md` §12; restore drill monthly (calendar-tracked).

Rollback: previous image kept warm (`:previous` tag); app rollback = redeploy previous; DB policy — migrations are roll-forward-only (no down migrations in prod); risky Phase-2 modules ship behind feature flags (`payroll.enabled` etc.) so rollback = flag off.

5. Deployment & go-live checklists

Every deploy: CI green incl. isolation+matrix suites · staging migration rehearsed · smoke passed · Sentry release created · rollback image verified present. **Go-live (pilot)**: prod env vars set + secrets rotated from defaults · seed run (company, roles, permissions, shifts, leave types, settings) · admin + HR users invited and verified · SMTP/Resend domain verified (SPF/DKIM) · S3 bucket private +

presign tested · backups running **and one restore tested** · rate limits enabled · audit logging verified end-to-end · month-lock and payroll flags in correct phase state · uptime + dead-man alerts firing to a human.

---

6. Build Order (canonical implementation sequence)

Phase 1:

1. **Repo scaffold** — Next.js 15 TS strict, Tailwind, shadcn/ui, ESLint/Prettier, Vitest. *(dep: —)*
2. **Docker + CI** — compose (pg/redis/minio/mailpit), GitHub Actions lint/type/test. *(1)*
3. **Prisma core schema migration** — org + auth + employees-min + audit tables, **RLS SQL, DB roles**, seed (permissions, roles, company, admin). *(2)*
4. **lib plumbing** — errors, envelope, `withApi`, tenantDb extension, events, queue, mailer, rate-limit. *(3)*
5. **Auth module** — login/logout/reset/invite endpoints + (auth) screens + session middleware. *(4)*
6. **RBAC** — permission catalog, `can()` / `requirePermission`, role assignment endpoints, sidebar-by-permission shell. *(5)*
7. **Audit plumbing** — event→audit writer, viewer API + `/admin/audit-logs`. *(6)*
8. **Org structure** — locations, departments, designations (API + admin/HR screens). *(6)*
9. **Employees** — CRUD, profile tabs, self-service edits, invite wiring, `/me/profile`. *(8)*
10. **Shifts** — definitions + assignments. *(9)*
11. **Attendance** — punches, check-in/out, calc job + worker boot, records views, corrections, month lock. *(10)*
12. **Leave** — types/policies/balances/accrual jobs, requests + approvals, `/me/leave`, approvals screens. *(11 for on_leave integration; 9)*
13. **Holidays** — CRUD + integration into calc/day-count. *(12)*
14. **Notifications** — service, templates, bell, center, announcements; wire P1 events + crons. *(12)*
15. **Dashboards** — employee `/dashboard`, `/team`, `/hr` + basic reports (R1–R4, R6–R7, R16). *(14)*
16. **Settings** — catalog service + `/admin/settings`, `/admin/company`. *(6)*
17. **Hardening** — permission-matrix + isolation + e2e suites, mobile pass, load test, backup/restore drill, pilot launch. *(all)*

Phase 2 order: 18. Payroll (engine → admin → runs → payslips) → 19. Documents → 20. Expenses → 21. Recruitment → 22. Onboarding → 23. Offboarding → 24. Performance → 25. Full reports/exports. Phase 3 order: 26. LMS → 27. Analytics → 28. Automation rules → 29. Integrations → 30. PWA/push → 31. Multi-tenant GA (self-serve signup, custom roles, SSO, pentest gate).

Decisions made in this document

- Roll-forward-only DB policy in prod (no down migrations) — rollback via app image + feature flags.
- Payroll go-live requires one **parallel run** (system vs existing manual payroll) with zero unexplained diffs.
- Permission-matrix tests are generated from the role matrix table, not hand-written, so drift between docs and code fails CI.
- 500-employee dataset is the standing performance fixture size.